

Java Method Overloading

Java Version: 1.0

Year: 1996

Feature: Method Overloading

UniqueClassName: MethodOverloading

1. Definition

Method overloading is a feature of Java in which **multiple methods can have the same method name within the same class, but must have different parameter lists.**

The parameter list can differ by:

1. **Number of parameters**
2. **Data types of parameters**
3. **Order of parameters**

The **return type alone cannot be used** to overload a method.

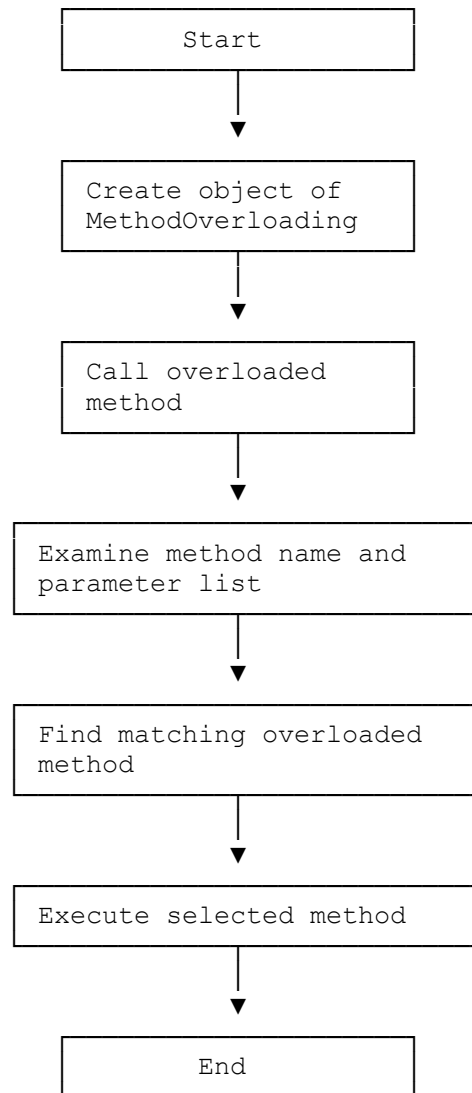
Method overloading is an example of **compile-time polymorphism** (static polymorphism).

2. General Syntax

```
class MethodOverloading {  
  
    void display() {  
        // Method 1  
    }  
  
    void display(int a) {  
        // Method 2  
    }  
  
    void display(int a, int b) {  
        // Method 3  
    }  
  
    void display(String name) {  
        // Method 4  
    }  
}
```

The compiler determines which display() method should be called based on the **arguments passed during compilation.**

3. Flowchart



4. Java Code Example

```
class MethodOverloading {  
  
    // Method with no parameter  
    void add() {  
        System.out.println("No parameters");  
    }  
  
    // Method with one int parameter  
    void add(int a) {  
        System.out.println("One integer parameter: " + a);  
    }  
  
    // Method with two int parameters  
    void add(int a, int b) {
```

```

        System.out.println("Two integer parameters: " + (a + b));
    }

    // Method with two double parameters
    void add(double a, double b) {
        System.out.println("Two double parameters: " + (a + b));
    }

    // Method with String parameter
    void add(String a) {
        System.out.println("String parameter: " + a);
    }

    public static void main(String[] args) {

        MethodOverloading obj = new MethodOverloading();

        obj.add();
        obj.add(10);
        obj.add(10, 20);
        obj.add(10.5, 20.5);
        obj.add("Java");
    }
}

```

Output

```

No parameters
One integer parameter: 10
Two integer parameters: 30
Two double parameters: 31.0
String parameter: Java

```

5. How the Compiler Selects the Method

When Java sees:

```
obj.add(10, 20);
```

it searches for methods named add and compares their parameter lists.

add()	→ 0 parameters	✗
add(int)	→ 1 parameter	✗
add(int, int)	→ 2 int	✓
add(double, double)	→ 2 double	✗
add(String)	→ 1 parameter	✗

Therefore:

```
add(int, int)
```

is selected.

Similarly:

```
obj.add(10.5, 20.5);
```

selects:

```
add(double, double)
```

6. Valid Ways to Overload a Method

A. Different number of parameters

```
void display(int a) {  
}
```

```
void display(int a, int b) {  
}
```

B. Different parameter types

```
void display(int a) {  
}
```

```
void display(double a) {  
}
```

C. Different order of parameter types

```
void display(int a, String b) {  
}
```

```
void display(String a, int b) {  
}
```

7. Invalid Method Overloading

Changing **only the return type** does not create method overloading.

```
class Test {
```

```

int calculate(int a) {
    return a;
}

// Compile-time error
double calculate(int a) {
    return a;
}
}

```

Both methods have exactly the same:

Method name: calculate

Parameters: (int)

Java does not consider the return type when distinguishing overloaded methods.

8. Key Characteristics

Feature	Method Overloading
Same method name	Yes
Same class	Usually yes
Different parameters required	Yes
Different number of parameters	Yes
Different parameter types	Yes
Different parameter order	Yes
Return type alone sufficient	No
Polymorphism type	Compile-time
Also called	Static polymorphism
Method selection	Compiler
Runtime overriding involved	No

Simple rule to remember

Same method name + different parameter list = Method Overloading

For example:

```
add()
add(int)
add(int, int)
add(double, double)
add(String)
```

All are valid overloaded versions of the add() method.

Java Constructors

Java Version: 1.0

Year: 1996

Feature: Constructors

UniqueClassName: Constructors

1. Definition

A **constructor** in Java is a special member of a class that is automatically invoked when an object is created.

Its primary purpose is to **initialize the newly created object**.

Important characteristics

- A constructor has the **same name as the class**.
 - It has **no return type**, not even void.
 - It is automatically called when an object is created using new.
 - A class can have **multiple constructors** through constructor overloading.
 - Constructors are **not inherited** by subclasses.
 - If no constructor is explicitly declared, the Java compiler can provide a **default constructor**.
 - Constructors can have access modifiers such as public, protected, package-private, and private.
-

2. Basic Syntax

```
class ClassName {

    ClassName() {
        // Initialization code
    }
}
```

Object creation:

```
ClassName object = new ClassName();
```

The sequence is:

```
new ClassName()
```

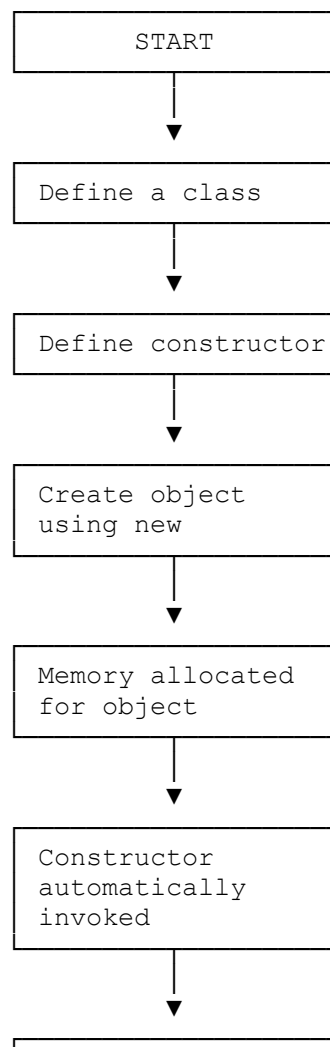


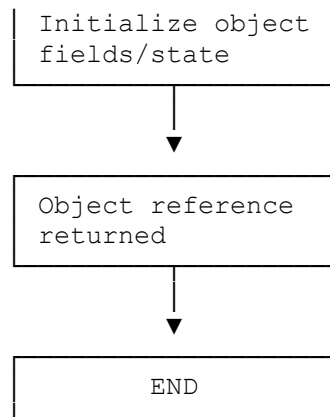
```
Constructor invoked
```



```
Object initialized
```

3. Constructor Flowchart





4. Java Code Example

```
class Constructors {  
  
    int id;  
    String name;  
  
    // Constructor  
    Constructors() {  
        id = 101;  
        name = "Anurag";  
    }  
  
    void display() {  
        System.out.println("ID   : " + id);  
        System.out.println("Name : " + name);  
    }  
  
    public static void main(String[] args) {  
  
        // Object creation  
        Constructors obj = new Constructors();  
  
        // Constructor is automatically called  
        obj.display();  
    }  
}
```

Output

```
ID       : 101  
Name     : Anurag
```

When this statement executes:

```
Constructors obj = new Constructors();
```

the constructor:

```
Constructors()
```

is automatically executed.

5. Parameterized Constructor

A constructor can accept parameters so that different objects can be initialized with different values.

```
class Constructors {  
  
    int id;  
    String name;  
  
    // Parameterized constructor  
    Constructors(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
  
    void display() {  
        System.out.println(id + " - " + name);  
    }  
  
    public static void main(String[] args) {  
  
        Constructors obj1 =  
            new Constructors(101, "Anurag");  
  
        Constructors obj2 =  
            new Constructors(102, "Rahul");  
  
        obj1.display();  
        obj2.display();  
    }  
}
```

Output

```
101 - Anurag  
102 - Rahul
```

Here:

```
new Constructors(101, "Anurag");
```

calls:

```
Constructors(int id, String name)
```

6. Constructor Overloading

A class can have more than one constructor as long as their parameter lists are different.

```
class Constructors {  
  
    int id;  
    String name;  
  
    // Constructor 1  
    Constructors() {  
        id = 0;  
        name = "Unknown";  
    }  
  
    // Constructor 2  
    Constructors(int id) {  
        this.id = id;  
        name = "Unknown";  
    }  
  
    // Constructor 3  
    Constructors(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
  
    void display() {  
        System.out.println(id + " - " + name);  
    }  
  
    public static void main(String[] args) {  
  
        Constructors obj1 = new Constructors();  
        Constructors obj2 = new Constructors(101);  
        Constructors obj3 = new Constructors(102, "Anurag");  
  
        obj1.display();  
        obj2.display();  
        obj3.display();  
    }  
}
```

Output:

0 - Unknown
101 - Unknown
102 - Anurag

This is called **constructor overloading**.

7. Constructor vs Method

Feature	Constructor	Method
Purpose	Initializes object	Performs an operation
Name	Must match class name	Can have any valid name
Return type	No return type	Has return type or void
Invocation	Automatically invoked during object creation	Usually explicitly invoked
Invocation syntax	new ClassName()	object.method()
Overloading	Yes	Yes
Inheritance	Not inherited	Can be inherited depending on access
Primary purpose	Object initialization	Behavior/operation

Example

Constructor:

```
Constructors() {  
    id = 101;  
}
```

Method:

```
void display() {
```

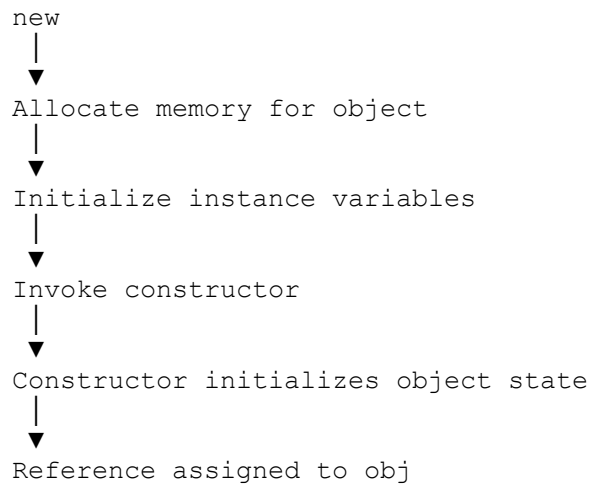
```
    System.out.println(id);  
}
```

8. Constructor Execution Sequence

Consider:

```
Constructors obj = new Constructors();
```

Conceptually, the process is:



So:

```
Constructors obj = new Constructors();
```

does **not simply mean "call the constructor."**

The `new` expression is responsible for creating the object, and the constructor is then invoked as part of object initialization.

9. `this()` and Constructor Chaining

One constructor can call another constructor in the same class using `this()`.

```
class Constructors {  
    int id;
```

```

String name;

Constructors() {
    this(101, "Unknown");
}

Constructors(int id, String name) {
    this.id = id;
    this.name = name;
}

void display() {
    System.out.println(id + " - " + name);
}

public static void main(String[] args) {
    Constructors obj = new Constructors();

    obj.display();
}
}

```

Output:

101 - Unknown

The flow is:

```

new Constructors()
    ↓
Constructors()
    ↓ this(101, "Unknown")
Constructors(int, String)
    ↓
Object initialized

```

Important rule

this() must be the **first statement** inside a constructor.

10. Default Constructor

If you don't declare **any constructor**, the compiler may provide a default constructor.

```

class Constructors {

    int id;
    String name;

    public static void main(String[] args) {

        Constructors obj = new Constructors();

        System.out.println(obj.id);
        System.out.println(obj.name);
    }
}

```

Output:

```

0
null

```

The default initialization comes from Java's object initialization rules.

However, once you explicitly declare a constructor:

```

Constructors(int id) {
    this.id = id;
}

```

the compiler does **not** additionally provide a no-argument constructor for you.

Therefore:

```

new Constructors();

```

would be a compilation error unless you explicitly define a no-argument constructor.

11. Key Rule to Remember

Constructor

↓

Same name as class

↓

No return type

↓

Called during object creation

↓

Initializes object

One-line definition

A constructor is a special class member that is automatically invoked during object creation to initialize the object's state.

Java this Keyword

Java Version: 1.0

Year: 1996

Feature: this Keyword

UniqueClassName: ThisKeyword

1. Definition

The **this keyword** in Java is a special reference variable that refers to the **current object**.

It is primarily used inside an instance method or constructor to refer to the object whose method or constructor is currently executing.

Simple definition

this refers to the current object of the current class.

For example:

this.id

means:

"The id instance variable belonging to the current object."

2. Why is this Needed?

Consider:

```
class ThisKeyword {  
  
    int id;  
  
    ThisKeyword(int id) {  
        id = id;  
    }  
}
```

```
}
```

Here both names are id:

Instance variable → id
Parameter → id

Inside:

```
id = id;
```

Java treats both references as the constructor parameter. The instance variable is not being explicitly referenced.

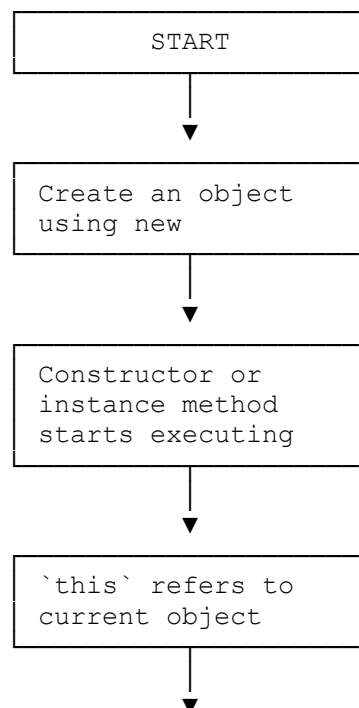
We can solve this using this:

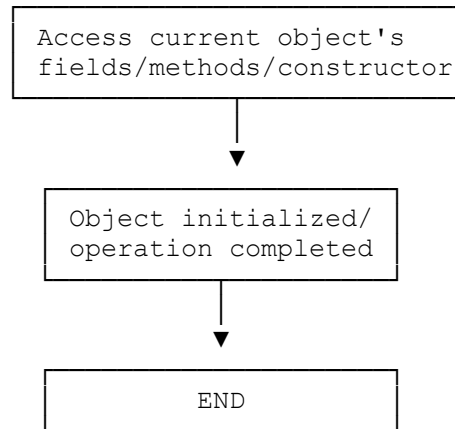
```
this.id = id;
```

Here:

this.id → instance variable
id → constructor parameter

3. Flowchart





4. Basic Java Code Example

```
class ThisKeyword {  
  
    int id;  
    String name;  
  
    ThisKeyword(int id, String name) {  
  
        this.id = id;  
        this.name = name;  
    }  
  
    void display() {  
  
        System.out.println("ID : " + this.id);  
        System.out.println("Name : " + this.name);  
    }  
  
    public static void main(String[] args) {  
  
        ThisKeyword obj = new ThisKeyword(101, "Anurag");  
  
        obj.display();  
    }  
}
```

Output

```
ID      : 101  
Name    : Anurag
```

5. How this Works

When this statement executes:

```
ThisKeyword obj = new ThisKeyword(101, "Anurag");
```

the constructor receives:

```
id  = 101  
name = "Anurag"
```

Inside the constructor:

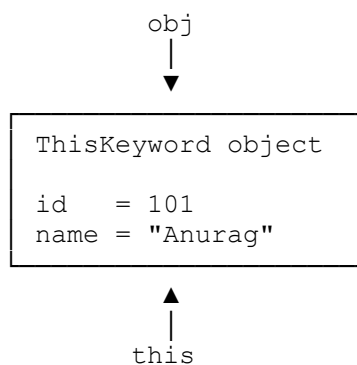
```
this.id = id;  
this.name = name;
```

means:

```
this.id    → object's instance variable  
id         → constructor parameter
```

```
this.name  → object's instance variable  
name       → constructor parameter
```

Conceptually:



So, during the execution of that constructor:

```
this == obj
```

conceptually refers to the same current object.

6. Main Uses of this

The this keyword has several important uses.

Use 1: Refer to the current object's instance variable

```
class ThisKeyword {  
  
    int id;  
  
    ThisKeyword(int id) {  
        this.id = id;  
    }  
  
    public static void main(String[] args) {  
  
        ThisKeyword obj = new ThisKeyword(100);  
  
        System.out.println(obj.id);  
    }  
}
```

Output:

100

Use 2: Resolve variable-name conflict

This is one of the most common uses.

```
class ThisKeyword {  
  
    int age;  
  
    ThisKeyword(int age) {  
        this.age = age;  
    }  
}
```

Here:

this.age

means the **instance variable**, while:

age

means the **constructor parameter**.

7. Calling the Current Class Method

this can also be used to explicitly call an instance method of the current object.

```
class ThisKeyword {  
  
    void message() {  
        System.out.println("Hello Java");  
    }  
  
    void display() {  
  
        this.message();  
    }  
  
    public static void main(String[] args) {  
  
        ThisKeyword obj = new ThisKeyword();  
  
        obj.display();  
    }  
}
```

Output:

Hello Java

Here:

this.message();

means:

Call message() on the current object.

The following is also valid:

message();

because Java implicitly understands the current object in an instance context.

8. Calling Another Constructor Using this()

this() is used to invoke **another constructor of the same class**.

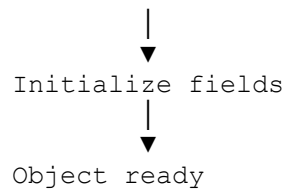
```
class ThisKeyword {  
  
    int id;  
    String name;  
  
    ThisKeyword() {  
  
        this(101, "Anurag");  
    }  
  
    ThisKeyword(int id, String name) {  
  
        this.id = id;  
        this.name = name;  
    }  
  
    void display() {  
  
        System.out.println(id + " - " + name);  
    }  
  
    public static void main(String[] args) {  
  
        ThisKeyword obj = new ThisKeyword();  
  
        obj.display();  
    }  
}
```

Output:

101 - Anurag

Flow:

```
new ThisKeyword()  
    |  
    ▼  
ThisKeyword()  
    |  
    ▼ this(101, "Anurag")  
ThisKeyword(int, String)
```



Important rule

this() must be the **first statement** in a constructor.

Correct:

```
ThisKeyword() {  
    this(101, "Anurag");  
}
```

Incorrect:

```
ThisKeyword() {  
  
    System.out.println("Hello");  
  
    this(101, "Anurag"); // Compilation error  
}
```

9. Passing this as an Argument

The current object can be passed to another method using this.

```
class ThisKeyword {  
  
    void display(ThisKeyword obj) {  
  
        System.out.println("Current object received");  
    }  
  
    void show() {  
  
        display(this);  
    }  
  
    public static void main(String[] args) {  
  
        ThisKeyword obj = new ThisKeyword();  

```

```
    obj.show();  
  }  
}
```

Output:

Current object received

Here:

```
display(this);
```

passes the current object to display().

10. Returning this

A method can return the current object using this.

```
class ThisKeyword {  
    ThisKeyword getObject() {  
        return this;  
    }  
  
    public static void main(String[] args) {  
        ThisKeyword obj = new ThisKeyword();  
        ThisKeyword result = obj.getObject();  
        System.out.println(obj == result);  
    }  
}
```

Output:

true

Because getObject() returns the same current object.

11. this and Multiple Objects

Suppose we create two objects:

```
ThisKeyword obj1 = new ThisKeyword(101);  
ThisKeyword obj2 = new ThisKeyword(102);
```

When obj1 calls a method:

```
obj1.display();
```

then:

this → obj1

When obj2 calls the same method:

```
obj2.display();
```

then:

this → obj2

Conceptually:

```
obj1.display()  
  ↓  
  this = obj1  
  
obj2.display()  
  ↓  
  this = obj2
```

Therefore, **this** changes according to the current object on which the instance method or constructor is executing.

12. Complete Example

```
class ThisKeyword {  
    int id;  
    String name;
```

```

// Constructor
ThisKeyword(int id, String name) {

    this.id = id;
    this.name = name;
}

// Instance method
void display() {

    System.out.println("ID    : " + this.id);
    System.out.println("Name  : " + this.name);

    this.showMessage();
}

// Another instance method
void showMessage() {

    System.out.println("Current object is being used.");
}

public static void main(String[] args) {

    ThisKeyword obj1 =
        new ThisKeyword(101, "Anurag");

    ThisKeyword obj2 =
        new ThisKeyword(102, "Rahul");

    obj1.display();
    obj2.display();
}
}

```

Output

```

ID      : 101
Name    : Anurag
Current object is being used.

```

```

ID      : 102
Name    : Rahul
Current object is being used.

```

13. this vs this()

These two are different.

this	this()
Refers to current object	Calls another constructor
Used to access instance members	Used for constructor chaining
Example: this.id	Example: this(101)
Represents an object reference	Represents a constructor invocation
Used in methods/constructors	Used only inside constructors

Example

`this.id = id;`

means:

Assign the parameter id to the current object's id.

Whereas:

`this(101);`

means:

Call another constructor of the same class with 101.

14. Important Rules

Rule	Explanation
this represents current object	It is the reference to the current object
this.id	Accesses current object's id
this.method()	Calls current object's instance method
this()	Calls another constructor in the same class
this(...) must be first	Constructor chaining must start with this(...)
Cannot use this in a static context	Static methods do not belong to a particular

	object
this is not a normal variable	It is a special reference provided by Java

For example, this is invalid:

```
class ThisKeyword {
    static void display() {
        System.out.println(this);
    }
}
```

because a static method does not have a current object.

15. Easy Way to Remember

```
this
├── this.variable
│   ↓
│   Current object's variable
├── this.method()
│   ↓
│   Current object's method
├── this()
│   ↓
│   Another constructor
├── method(this)
│   ↓
│   Pass current object
├── return this
│   ↓
│   Return current object
```

One-line definition

this is a Java reference that represents the current object and is commonly used to access the current object's instance members, invoke its methods, perform constructor chaining, pass the current object as an argument, or return the current object.

Java static Keyword

Java Version: 1.0

Year: 1996

Feature: static Keyword

UniqueClassName: StaticKeyword

1. Definition

The **static keyword** in Java is used to define a member that belongs to the **class itself rather than to individual objects**.

A static member is associated with the class and is generally created/shared at the class level.

Simple definition

static means the member belongs to the class, not to each individual object.

For example:

```
class StaticKeyword {  
    static int count;  
}
```

Here, count belongs to the StaticKeyword class and is shared by its objects.

2. Why is static Used?

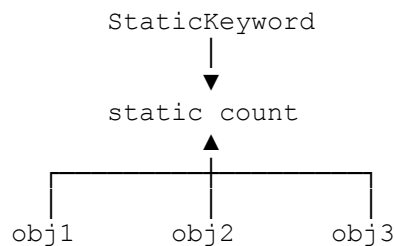
Without static, every object gets its own copy of an instance variable.

Object 1 → count

Object 2 → count

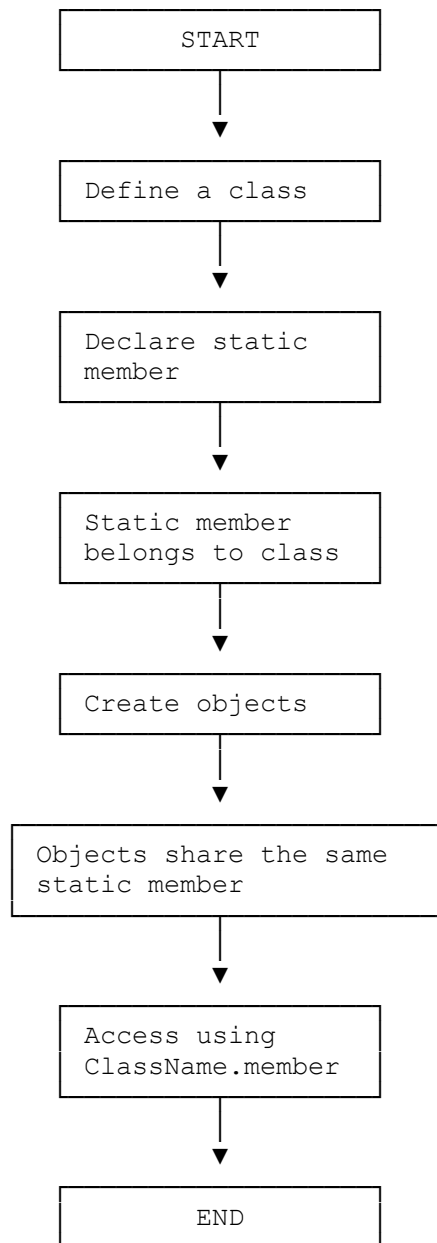
Object 3 → count

With static, there is one class-level variable:



Therefore, static is useful when a value or behavior should be **shared across all objects of a class**.

3. Flowchart



4. Types of static Members

The static keyword can be used with:

1. **Static variables**
2. **Static methods**
3. **Static blocks**
4. **Static nested classes**

Example:

```
class StaticKeyword {  
    static int count;          // Static variable  
  
    static void display() { // Static method  
        System.out.println(count);  
    }  
  
    static {                  // Static block  
        count = 10;  
    }  
  
    static class Helper {     // Static nested class  
    }  
}
```

5. Static Variable

A static variable is also called a:

- **Class variable**
- **Static field**

It belongs to the class rather than individual objects.

Java Example

```
class StaticKeyword {  
  
    static int count = 0;  
  
    StaticKeyword() {
```

```
        count++;
    }

    public static void main(String[] args) {

        StaticKeyword obj1 = new StaticKeyword();
        StaticKeyword obj2 = new StaticKeyword();
        StaticKeyword obj3 = new StaticKeyword();

        System.out.println("Objects created: " + count);
    }
}
```

Output

Objects created: 3

What happens?

Initially:

count = 0

After obj1:

count = 1

After obj2:

count = 2

After obj3:

count = 3

There is only **one shared count variable**.

6. Static Variable vs Instance Variable

```
class StaticKeyword {

    int instanceCount = 0;

    static int staticCount = 0;

    StaticKeyword() {
```

```

        instanceCount++;
        staticCount++;
    }

    public static void main(String[] args) {

        StaticKeyword obj1 = new StaticKeyword();
        StaticKeyword obj2 = new StaticKeyword();

        System.out.println(obj1.instanceCount);
        System.out.println(obj2.instanceCount);

        System.out.println(StaticKeyword.staticCount);
    }
}

```

Output:

```

1
1
2

```

Why?

Instance variable:

```

obj1 → instanceCount = 1
obj2 → instanceCount = 1

```

Static variable:

```

Class → staticCount = 2

```

7. Static Method

A method declared using static is called a **static method** or **class method**.

Example:

```

class StaticKeyword {

    static void display() {

        System.out.println("Static method executed.");
    }
}

```

```
public static void main(String[] args) {  
    StaticKeyword.display();  
}  
}
```

Output:

Static method executed.

Notice that we did not create an object.

We directly used:

```
StaticKeyword.display();
```

The general syntax is:

```
ClassName.staticMethod();
```

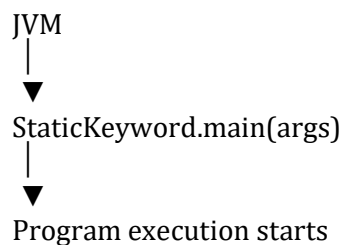
8. Why Is main() Static?

The standard Java entry point is:

```
public static void main(String[] args)
```

The main() method is static because the JVM needs to invoke it **without first creating an object of the application class**.

Conceptually:



If main() were an ordinary instance method, an object would have to exist before it could be invoked.

9. Static Method Restrictions

A static method can directly access **static members**.

```
class StaticKeyword {  
    static int number = 100;  
  
    static void display() {  
        System.out.println(number);  
    }  
  
    public static void main(String[] args) {  
        display();  
    }  
}
```

This works because both are static.

However, a static method cannot directly access an instance variable:

```
class StaticKeyword {  
    int number = 100;  
  
    static void display() {  
        // Compilation error  
        // System.out.println(number);  
    }  
}
```

Why?

```
static method  
↓  
belongs to class  
↓  
no particular object  
↓  
cannot directly access instance member
```

An object can be used:

```
class StaticKeyword {  
    int number = 100;
```

```
static void display() {  
    StaticKeyword obj = new StaticKeyword();  
    System.out.println(obj.number);  
}  
  
public static void main(String[] args) {  
    display();  
}  
}
```

10. Static Block

A **static block** is a block of code declared using static.

```
class StaticKeyword {  
    static int number;  
  
    static {  
        number = 100;  
        System.out.println("Static block executed.");  
    }  
  
    public static void main(String[] args) {  
        System.out.println("Number: " + number);  
    }  
}
```

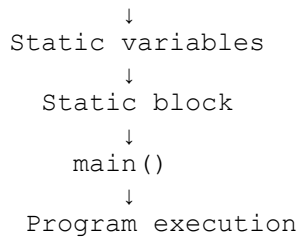
Output:

```
Static block executed.  
Number: 100
```

The static block is executed when the class is initialized, before the main() method executes in this example.

Flow

Class loading/initialization



11. Multiple Static Blocks

A class can have multiple static blocks.

```
class StaticKeyword {  
    static {  
        System.out.println("Static Block 1");  
    }  
    static {  
        System.out.println("Static Block 2");  
    }  
    public static void main(String[] args) {  
        System.out.println("Main method");  
    }  
}
```

Output:

```
Static Block 1  
Static Block 2  
Main method
```

Static blocks execute in their **textual order** during class initialization.

12. Static Nested Class

Java also allows a nested class to be declared static.

```

class StaticKeyword {

    static class Helper {

        void display() {
            System.out.println("Static nested class");
        }
    }

    public static void main(String[] args) {

        StaticKeyword.Helper obj =
            new StaticKeyword.Helper();

        obj.display();
    }
}

```

Output:

Static nested class

The nested class can be accessed using:

OuterClass.NestedClass

13. Complete Java Code Example

```

class StaticKeyword {

    // Static variable
    static int objectCount = 0;

    // Instance variable
    int id;

    // Static block
    static {
        System.out.println("Static block executed.");
    }

    // Constructor
    StaticKeyword(int id) {

        this.id = id;
        objectCount++;
    }
}

```

```

// Static method
static void displayObjectCount() {

    System.out.println(
        "Total objects: " + objectCount
    );
}

// Instance method
void displayId() {

    System.out.println("Object ID: " + id);
}

public static void main(String[] args) {

    StaticKeyword obj1 =
        new StaticKeyword(101);

    StaticKeyword obj2 =
        new StaticKeyword(102);

    obj1.displayId();
    obj2.displayId();

    StaticKeyword.displayObjectCount();
}
}

```

Output

```

Static block executed.
Object ID      : 101
Object ID      : 102
Total objects   : 2

```

14. Memory Concept

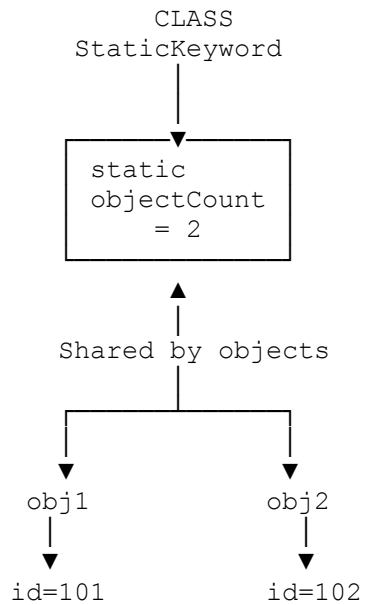
Suppose:

```

StaticKeyword obj1 = new StaticKeyword(101);
StaticKeyword obj2 = new StaticKeyword(102);

```

Conceptually:



The id variable is separate for each object, while objectCount is shared.

15. static vs Non-Static

Feature	static	Non-static
Belongs to	Class	Object
Also called	Class member	Instance member
Object required for access	Usually no	Yes
Number of copies	Generally one per class initialization	One per object
Access	ClassName.member	object.member
Can directly access static members	Yes	Yes
Can directly access instance members	No	Yes
Can use this	No	Yes

Example	static int count	int id
----------------	------------------	--------

16. Important Rules

Rule 1 — Static member belongs to the class

StaticKeyword.objectCount

is the preferred form for accessing a static field.

Rule 2 — Static methods do not have this

This is invalid:

```
static void display() {
    System.out.println(this);
}
```

because this represents a particular object, while a static method belongs to the class.

Rule 3 — Static methods can directly access static members

```
static int count;

static void display() {
    System.out.println(count);
}
```

Rule 4 — Static methods cannot directly access instance members

```
int id;

static void display() {
    // System.out.println(id); // Error
}
```

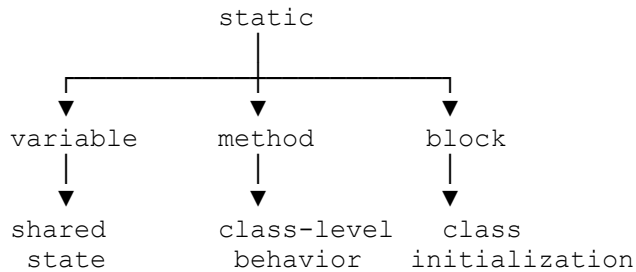
Rule 5 — Static members are shared

If one object changes a static field:

`obj1.count = 100;`

other objects accessing that same class-level field see the changed value.

17. Easy Way to Remember



One-line definition

The static keyword makes a field, method, initializer block, or nested class a class-level member rather than an ordinary instance member.

Java final Keyword

Java Version: 1.0

Year: 1996

Feature: final Keyword

UniqueClassName: FinalKeyword

1. Definition

The **final keyword** in Java is used to restrict modification, overriding, or inheritance, depending on where it is applied.

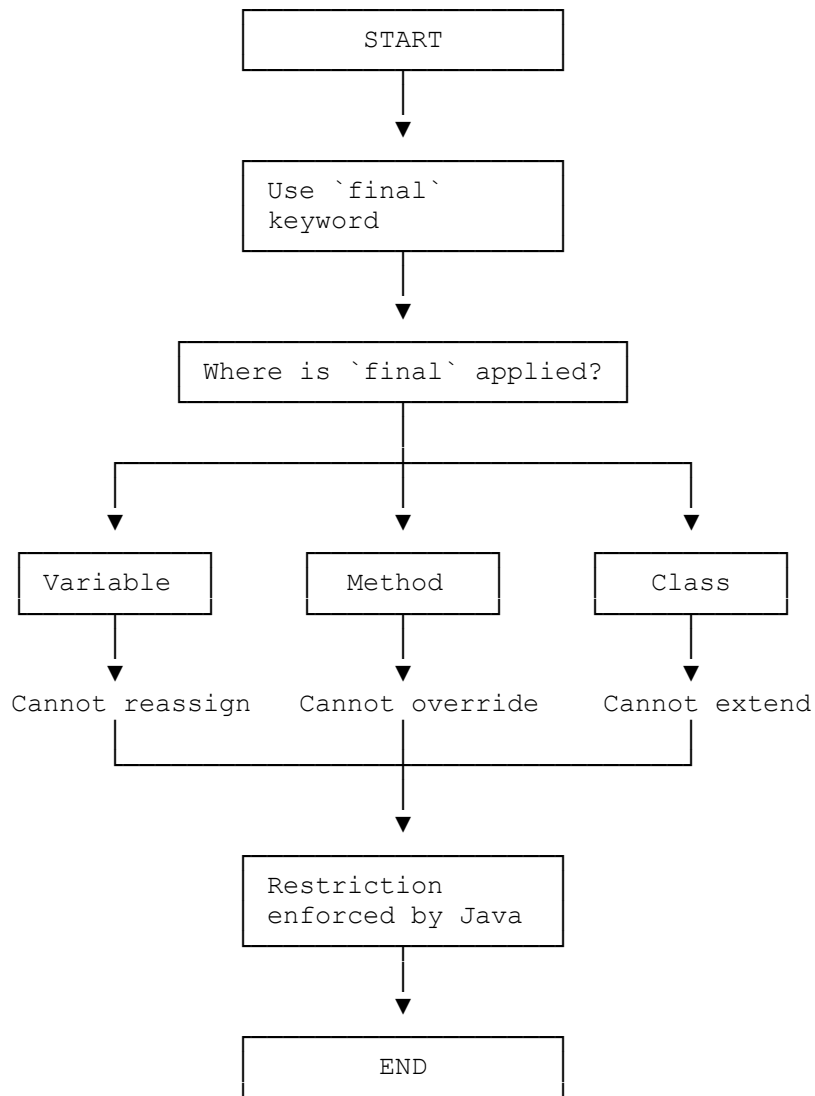
It can be used with:

1. **Variables** → value/reference cannot be reassigned
2. **Methods** → method cannot be overridden
3. **Classes** → class cannot be extended
4. **Parameters** → parameter cannot be reassigned inside the method

Simple definition

final means "cannot be changed further" in the particular context where it is applied.

2. Flowchart



3. final Variable

When a variable is declared final, it can be **assigned only once**.

```
final int MAX = 100;
```

After assignment:

```
MAX = 200; // Compilation error
```

Example

```
class FinalKeyword {  
    public static void main(String[] args) {  
        final int MAX_VALUE = 100;  
        System.out.println("Maximum value: " + MAX_VALUE);  
        // MAX_VALUE = 200; // Compilation error  
    }  
}
```

Output:

Maximum value: 100

4. Final Variable Must Be Initialized

A final variable must receive a value before it is used.

```
final int number;
```

This declaration itself is allowed.

You can assign it later:

```
number = 100;
```

But you cannot assign it again:

```
number = 200; // Compilation error
```

Example:

```
class FinalKeyword {  
    public static void main(String[] args) {  
        final int number;  
        number = 100;  
        System.out.println(number);  
    }  
}
```

```
        // number = 200; // Error
    }
}
```

5. Final Instance Variable

A final instance variable can be initialized in a constructor.

```
class FinalKeyword {
    final int id;

    FinalKeyword(int id) {
        this.id = id;
    }

    void display() {
        System.out.println("ID: " + id);
    }

    public static void main(String[] args) {
        FinalKeyword obj =
            new FinalKeyword(101);

        obj.display();
    }
}
```

Output:

ID: 101

Here:

```
final int id;
```

cannot be assigned again after initialization.

6. Final Reference Variable

A very important point is that final on an object reference means the **reference cannot point to another object**.

It does **not necessarily make the object itself immutable**.

Example:

```
class FinalKeyword {  
    public static void main(String[] args) {  
        final StringBuilder builder =  
            new StringBuilder("Java");  
  
        builder.append(" Programming");  
  
        System.out.println(builder);  
  
        // builder = new StringBuilder("Python");  
        // Compilation error  
    }  
}
```

Output:

Java Programming

The reference:

builder

cannot point to another StringBuilder, but the object it references can still be modified.

Conceptually:

final reference



StringBuilder object "Java Programming"
--

7. final Method

A method declared final **cannot be overridden by a subclass**.

Example:

```

class Parent {

    final void display() {

        System.out.println("Parent display()");

    }

}

class Child extends Parent {

    // Compilation error
    // void display() {
    //     System.out.println("Child display()");
    // }

}

class FinalKeyword {

    public static void main(String[] args) {

        Child obj = new Child();

        obj.display();

    }

}

```

Output:

Parent display()

The subclass can inherit the method, but it cannot provide another implementation of that method through overriding.

8. Final Class

A class declared final **cannot be extended**.

```

final class Parent {

    void display() {

        System.out.println("Parent class");

    }

}

```

This is invalid:

```

class Child extends Parent {

```

```
}
```

because Parent is final.

Example

```
final class FinalClass {  
    void display() {  
        System.out.println("Final class");  
    }  
}  
  
class FinalKeyword {  
    public static void main(String[] args) {  
        FinalClass obj = new FinalClass();  
        obj.display();  
    }  
}
```

Output:

Final class

9. Final Method + Inheritance

A final method can be inherited, but it cannot be overridden.

```
class Parent {  
    final void show() {  
        System.out.println("Parent show()");  
    }  
}  
  
class Child extends Parent {  
    void display() {  
        System.out.println("Child display()");  
    }  
}
```

```

class FinalKeyword {

    public static void main(String[] args) {

        Child obj = new Child();

        obj.show();
        obj.display();
    }
}

```

Output:

Parent show()
Child display()

So:

```

final method
├── Can be inherited      → Yes
└── Can be overridden    → No

```

10. Final Method Parameter

A method parameter can also be declared final.

```

class FinalKeyword {

    static void calculate(final int number) {

        System.out.println(number);

        // number = 200; // Compilation error
    }

    public static void main(String[] args) {

        calculate(100);
    }
}

```

Output:

100

The parameter number cannot be reassigned inside the method.

11. Final Local Variable

A local variable can also be final.

```
class FinalKeyword {  
    public static void main(String[] args) {  
        final int age = 25;  
        System.out.println("Age: " + age);  
        // age = 30; // Compilation error  
    }  
}
```

12. static final Constant

static and final are frequently used together to define constants.

```
class FinalKeyword {  
    static final double PI = 3.141592653589793;  
    public static void main(String[] args) {  
        System.out.println("PI = " + PI);  
    }  
}
```

Output:

PI = 3.141592653589793

The meaning is:

static
↓
belongs to the class

final



cannot be reassigned

static final



class-level constant

Java conventionally uses **UPPER_SNAKE_CASE** for constants:

```
static final int MAX_SIZE = 100;
static final double PI = 3.14159;
static final String APPLICATION_NAME = "MyApp";
```

13. Complete Java Code Example

```
class FinalKeyword {

    // Final static constant
    static final int MAX_USERS = 100;

    // Final instance variable
    final int id;

    // Constructor
    FinalKeyword(int id) {

        this.id = id;
    }

    // Final method
    final void display() {

        System.out.println("ID: " + id);
        System.out.println("Maximum users: " + MAX_USERS);
    }

    public static void main(String[] args) {

        FinalKeyword obj =
            new FinalKeyword(101);

        obj.display();

        // obj.id = 102;          // Compilation error
        // MAX_USERS = 200;      // Compilation error
    }
}
```

Output

ID : 101
Maximum users : 100

14. final vs static final

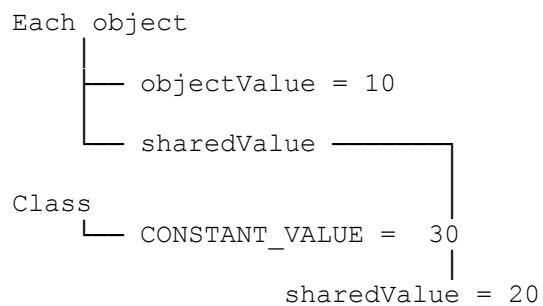
These are not the same.

Declaration	Meaning
final int x	Cannot reassign x; normally each object has its own variable
static int x	One class-level shared variable; can be changed
static final int x	One class-level variable that cannot be reassigned

Example:

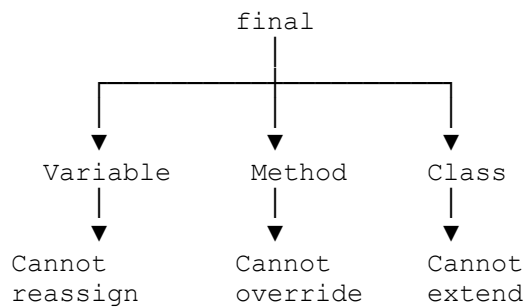
```
class Example {  
    final int objectValue = 10;  
    static int sharedValue = 20;  
    static final int CONSTANT_VALUE = 30;  
}
```

Conceptually:



15. Three Major Uses of final

The easiest way to remember final is:



Variable

```
final int x = 10;
```

Cannot reassign x.

Method

```
final void display() {  
}
```

Cannot override display().

Class

```
final class A {  
}
```

Cannot extend A.

16. final vs finally vs finalize()

These three terms are completely different.

Term	Meaning
final	Keyword used for variables, methods, classes, etc.
finally	Block associated with exception handling

finalize()	Historical Object method related to GC cleanup; deprecated since Java 9 and subject to removal
-------------------	--

Example:

```
final int x = 10;
```

is related to restriction.

```
try {
    // code
} finally {
    // cleanup
}
```

is related to exception handling.

For modern Java, **do not use finalize() for resource cleanup**; use constructs such as try-with-resources and AutoCloseable.

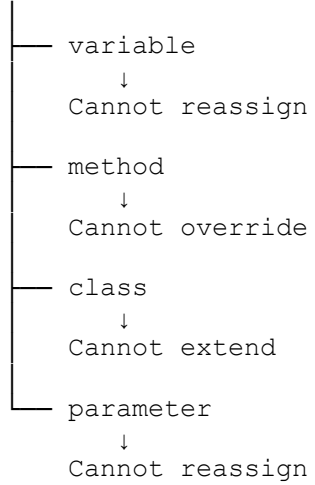
17. Important Rules

Rule	Explanation
final variable	Cannot be reassigned after initialization
final reference	Reference cannot point to another object
Final object's state	May still be mutable
final method	Cannot be overridden
final class	Cannot be extended
final parameter	Cannot be reassigned inside method
static final	Commonly used for constants
Final variable initialization	Must happen before use
Constructor can initialize final field	Yes, when appropriate
Final class can have subclasses	No

--	--

18. Easy Way to Remember

final



One-line definition

The final keyword restricts further modification: a final variable cannot be reassigned, a final method cannot be overridden, and a final class cannot be extended.

Java Access Modifiers

Java Version: 1.0

Year: 1996

Feature: Access Modifiers

UniqueClassName: AccessModifiers

1. Definition

Access modifiers in Java are keywords that control the **visibility and accessibility** of classes, constructors, methods, fields, and other members.

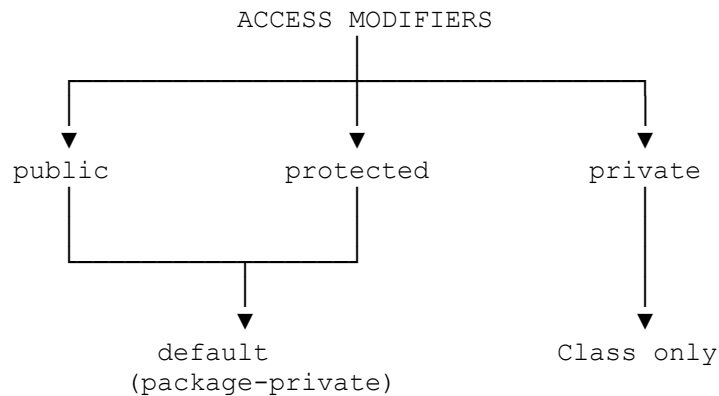
Java provides four access levels:

1. public
2. protected
3. **default** / package-private — when no modifier is specified
4. private

Simple definition

Access modifiers determine where a Java class or class member can be accessed from.

2. Four Access Levels



A more useful accessibility hierarchy is:

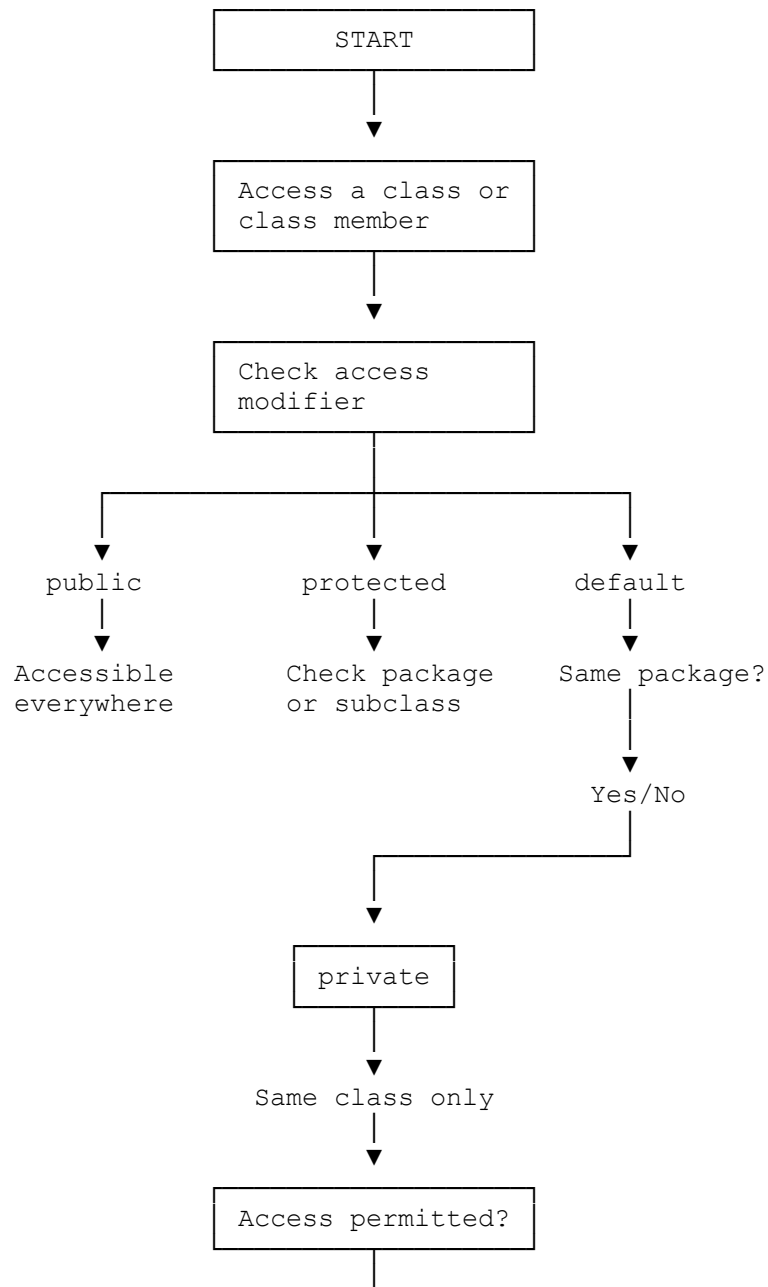
```
public
↓
Everywhere

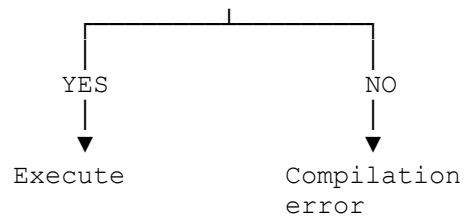
protected
↓
Same package + subclasses in other packages
```

default
↓
Same package only

private
↓
Same class only

3. Flowchart





4. Access Modifier Comparison

Access Modifier	Same Class	Same Package	Subclass in Different Package	Different Package, Non-Subclass
public	✓	✓	✓	✓
protected	✓	✓	✓*	✗
default	✓	✓	✗	✗
private	✓	✗	✗	✗

* A protected member in another package is accessible to a subclass through inheritance rules; it is not generally accessible through an arbitrary superclass object reference.

5. public Access Modifier

The public modifier provides the **widest accessibility**.

A public member can generally be accessed from any package, provided the containing types and other access conditions permit it.

Example

```

package package1;

public class AccessModifiers {

    public int number = 100;

    public void display() {
        System.out.println("Public method");
    }
}
  
```

```
}  
}
```

Another package can access it:

```
package package2;  
  
import package1.AccessModifiers;  
  
class Test {  
  
    public static void main(String[] args) {  
  
        AccessModifiers obj = new AccessModifiers();  
  
        System.out.println(obj.number);  
  
        obj.display();  
    }  
}
```

Output:

```
100  
Public method
```

6. private Access Modifier

A private member is accessible **only within the class that declares it**.

```
class AccessModifiers {  
  
    private int number = 100;  
  
    private void display() {  
        System.out.println("Private method");  
    }  
  
    public void show() {  
  
        System.out.println(number);  
  
        display();  
    }  
}
```

Here, number and display() can be accessed inside AccessModifiers.

But this is invalid:

```
class Test {  
    public static void main(String[] args) {  
        AccessModifiers obj = new AccessModifiers();  
  
        // obj.number; // Compilation error  
        // obj.display(); // Compilation error  
    }  
}
```

Concept

```
AccessModifiers  
├── private number  
│   ↓  
│   Class only  
└── private display()  
    ↓  
    Class only
```

7. Default Access Modifier

When no access modifier is specified, Java uses **package-private (default) access**.

Example:

```
class AccessModifiers {  
    int number = 100;  
  
    void display() {  
        System.out.println("Default access");  
    }  
}
```

This is equivalent to package-level accessibility:

Same class → ✓

Same package → ✓
Different package → ✗

Example:

```
package package1;

class AccessModifiers {

    int number = 100;

    void display() {
        System.out.println(number);
    }
}
```

Another class in the **same package** can access it:

```
package package1;

class Test {

    public static void main(String[] args) {

        AccessModifiers obj =
            new AccessModifiers();

        System.out.println(obj.number);

        obj.display();
    }
}
```

8. protected Access Modifier

A protected member is accessible:

1. Within the same class
2. Within the same package
3. In subclasses, including subclasses in another package, subject to Java's protected-access rules

Example:

```
class AccessModifiers {
```

```
protected int number = 100;

protected void display() {

    System.out.println("Protected method");
}
}
```

A subclass can access it:

```
class Child extends AccessModifiers {

    void show() {

        System.out.println(number);

        display();
    }
}
```

9. protected Across Different Packages

This is an important interview topic.

Package 1

```
package package1;

public class Parent {

    protected int number = 100;

    protected void display() {

        System.out.println("Protected method");
    }
}
```

Package 2

```
package package2;

import package1.Parent;
```

```
public class Child extends Parent {  
  
    public void show() {  
  
        System.out.println(number);  
  
        display();  
    }  
}
```

This is valid because Child is a subclass of Parent.

However, code in an unrelated class in package2 cannot simply access Parent's protected member as if it were public.

10. Complete Java Code Example

```
class AccessModifiers {  
  
    // Public member  
    public int publicNumber = 10;  
  
    // Protected member  
    protected int protectedNumber = 20;  
  
    // Default member  
    int defaultNumber = 30;  
  
    // Private member  
    private int privateNumber = 40;  
  
    public void display() {  
  
        System.out.println(  
            "Public      : " + publicNumber  
        );  
  
        System.out.println(  
            "Protected : " + protectedNumber  
        );  
  
        System.out.println(  
            "Default   : " + defaultNumber  
        );  
  
        System.out.println(  
            "Private   : " + privateNumber  
        );  
    }  
}
```

```

public static void main(String[] args) {

    AccessModifiers obj =
        new AccessModifiers();

    obj.display();

    System.out.println(
        "Public: " + obj.publicNumber
    );

    System.out.println(
        "Protected: " + obj.protectedNumber
    );

    System.out.println(
        "Default: " + obj.defaultNumber
    );

    // System.out.println(obj.privateNumber);
    // Compilation error
}
}

```

Output

```

Public          : 10
Protected       : 20
Default         : 30
Private         : 40

```

```

Public          : 10
Protected       : 20
Default         : 30

```

11. Access Modifiers for Classes

There is an important distinction between **top-level classes** and **members/nested classes**.

A top-level class can generally be:

```

public class AccessModifiers {
}

```

or package-private:

```
class AccessModifiers {  
}
```

A top-level class **cannot** be declared:

```
private class AccessModifiers {  
}
```

or:

```
protected class AccessModifiers {  
}
```

Those modifiers are not permitted for a top-level class.

Top-Level Class

Modifier	Allowed?
public	✓
protected	✗
private	✗
default/package-private	✓

12. Nested Classes

Nested classes can use all four access levels.

```
class Outer {  
  
    public class PublicNested {  
    }  
  
    protected class ProtectedNested {  
    }  
  
    class DefaultNested {  
    }  
  
    private class PrivateNested {  
    }  
}
```

```
}
```

So the restriction on private and protected applies specifically to **top-level classes**, not nested classes.

13. Access Modifiers with Constructors

Constructors can also have access modifiers.

Public constructor

```
public AccessModifiers() {  
}
```

Object can generally be created from accessible code.

Protected constructor

```
protected AccessModifiers() {  
}
```

Accessible according to protected access rules.

Default constructor

```
AccessModifiers() {  
}
```

Accessible within the same package.

Private constructor

```
private AccessModifiers() {  
}
```

Accessible only within the declaring class.

A private constructor is commonly used in patterns such as a singleton or utility-style class design.

14. Private Constructor Example

```
class AccessModifiers {  
    private AccessModifiers() {  
        System.out.println(  
            "Private constructor called"  
        );  
    }  
  
    public static AccessModifiers createObject() {  
        return new AccessModifiers();  
    }  
  
    public static void main(String[] args) {  
        AccessModifiers obj =  
            AccessModifiers.createObject();  
    }  
}
```

Output:

Private constructor called

Direct external construction would not be allowed:

```
// AccessModifiers obj = new AccessModifiers();
```

because the constructor is private.

15. Access Modifiers and Encapsulation

Access modifiers are fundamental to **encapsulation**.

For example:

```
class Account {  
    private double balance;
```

```

public void deposit(double amount) {

    if (amount > 0) {
        balance += amount;
    }
}

public double getBalance() {

    return balance;
}
}

```

Here:

```

balance
↓
private
↓
Direct external modification prevented
↓
public methods control access
↓
Encapsulation

```

This is one of the most important practical uses of access modifiers.

16. Common Pattern

A common Java design is:

```

class User {

    private String name;
    private int age;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getAge() {

```

```

    return age;
}

public void setAge(int age) {
    this.age = age;
}
}

```

Here:

```

private fields
  ↓
Hide internal state
  ↓
public methods
  ↓
Controlled access

```

This is the basis of **data hiding and encapsulation**.

17. Access Modifier Hierarchy

From **most accessible** to **least accessible**:

```

public
  ↓
protected
  ↓
default / package-private
  ↓
private

```

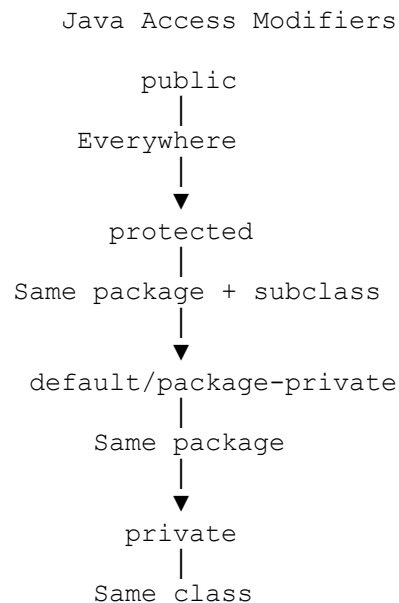
However, protected is not simply "between public and default" in every practical scenario because its access in other packages is tied specifically to inheritance.

18. Important Rules

Rule	Explanation
------	-------------

public	Widest access
protected	Same package + qualifying subclass access
default	Same package only
private	Same declaring class only
Top-level class	Can be public or package-private
Top-level private class	✗ Not allowed
Top-level protected class	✗ Not allowed
Nested class	Can use all four access levels
Private fields	Commonly used for encapsulation
Public methods	Commonly used to expose controlled operations

19. Easy Way to Remember



One-line definition

Access modifiers are Java keywords that control the visibility and accessibility of classes, constructors, fields, methods, and nested classes.

Java Packages and import

Java Version: 1.0

Year: 1996

Feature: Packages / import

UniqueClassName: PackagesImport

1. Definition

A **package** in Java is a mechanism used to **group related classes, interfaces, enums, and other types** under a common namespace.

The import statement is used to allow a Java source file to **refer to accessible types from another package without repeatedly writing their fully qualified names**.

Simple definition

Package = organizes related Java types into namespaces.

import = allows convenient use of types from another package.

For example:

```
import java.util.ArrayList;
```

allows us to write:

```
ArrayList<String> list = new ArrayList<>();
```

instead of:

```
java.util.ArrayList<String> list =  
    new java.util.ArrayList<>();
```

2. Why Are Packages Used?

Packages provide several important benefits:

1. **Organization** of related classes
2. **Namespace management**
3. **Avoiding class-name conflicts**
4. **Access control**
5. **Better project structure**
6. **Code maintainability**
7. **Reusable libraries and APIs**

For example:

```
com.company.employee  
com.company.customer  
com.company.payment  
com.company.order
```

Each package can contain related classes.

3. Package Structure

Suppose we have:

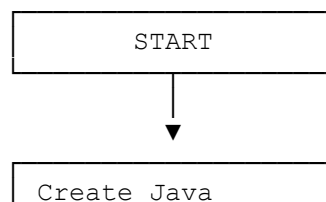
```
src/  
└── com/  
    └── example/  
        └── employee/  
            ├── Employee.java  
            └── EmployeeService.java
```

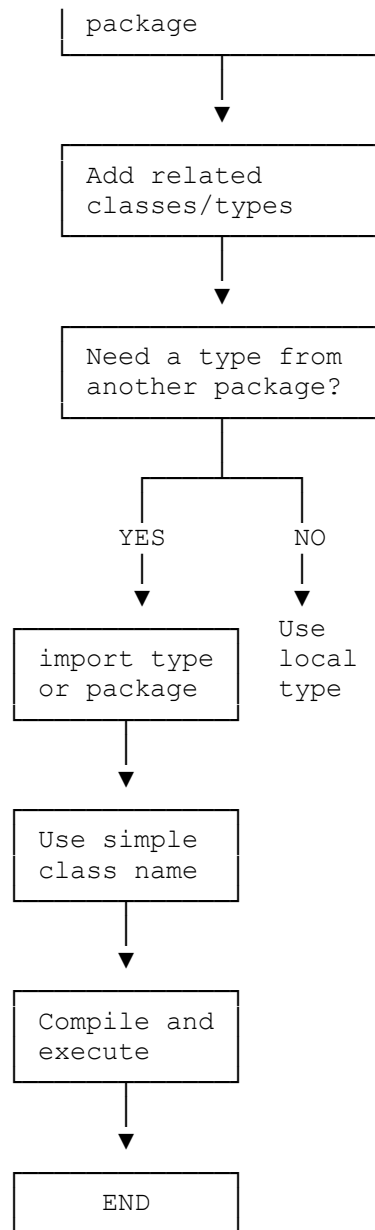
The Employee.java file can begin with:

```
package com.example.employee;
```

The package declaration establishes the class's package.

4. Flowchart





5. Basic Package Example

File 1: Employee.java

```
package com.example.employee;  
  
public class Employee {  
    public void display() {
```

```
        System.out.println(  
            "Employee class from employee package"  
        );  
    }  
}
```

The important statement is:

```
package com.example.employee;
```

This places Employee in the:

```
com.example.employee
```

```
package.
```

6. Using import

Suppose another class is in:

```
com.example.app
```

File 2: PackagesImport.java

```
package com.example.app;  
  
import com.example.employee.Employee;  
  
public class PackagesImport {  
    public static void main(String[] args) {  
        Employee employee = new Employee();  
        employee.display();  
    }  
}
```

Output

```
Employee class from employee package
```

The import statement:

```
import com.example.employee.Employee;
```

allows us to use:

Employee

instead of:

```
com.example.employee.Employee
```

7. Complete Directory Structure

```
PackagesImportProject/  
├── src/  
│   ├── com/  
│   │   └── example/  
│   │       ├── employee/  
│   │       │   └── Employee.java  
│   │       └── app/  
│   │           └── PackagesImport.java
```

Employee.java

```
package com.example.employee;
```

```
public class Employee {  
    public void display() {  
        System.out.println(  
            "Employee package"  
        );  
    }  
}
```

PackagesImport.java

```
package com.example.app;
```

```
import com.example.employee.Employee;
```

```
public class PackagesImport {  
    public static void main(String[] args) {  
        Employee employee = new Employee();  
        employee.display();  
    }  
}
```

8. Fully Qualified Class Name

Every class has a **fully qualified class name (FQCN)** consisting of:

Package name + Class name

For example:

com.example.employee.Employee

Here:

```
com.example.employee → package  
Employee              → class
```

Without import, we can use the fully qualified name directly:

```
class PackagesImport {  
    public static void main(String[] args) {  
        com.example.employee.Employee employee =  
            new com.example.employee.Employee();  
        employee.display();  
    }  
}
```

This is valid, although importing the class is generally more readable.

9. Importing a Single Class

You can import a specific class:

```
import java.util.ArrayList;
```

Then:

```
ArrayList<String> list = new ArrayList<>();
```

10. Importing Multiple Classes

```
import java.util.ArrayList;  
import java.util.HashMap;  
import java.util.List;
```

Then:

```
List<String> names = new ArrayList<>();
```

```
HashMap<Integer, String> employees =  
    new HashMap<>();
```

11. Wildcard Import

Java allows importing all accessible types in a package using `*`.

```
import java.util.*;
```

Then classes such as these can be referenced by simple name:

```
ArrayList  
HashMap  
HashSet  
LinkedList  
Scanner
```

For example:

```
import java.util.*;

class PackagesImport {

    public static void main(String[] args) {

        ArrayList<String> names =
            new ArrayList<>();

        names.add("Anurag");
        names.add("Rahul");

        System.out.println(names);
    }
}
```

Output:

[Anurag, Rahul]

Important

```
import java.util.*;
```

does **not** recursively import classes from subpackages.

For example, it does not mean:

```
java.util.concurrent.*
```

is automatically imported.

12. import Does Not Import a Package's Subpackages

Suppose we have:

```
com.example
├── employee
└── customer
```

This:

```
import com.example.*;
```

does not automatically import classes from:

com.example.employee
com.example.customer

Packages are separate namespaces.

13. Static Import

Java also supports **static imports**.

A static import allows static fields and static methods to be referenced without qualifying them with the class name.

Example:

```
import static java.lang.Math.PI;
import static java.lang.Math.sqrt;

class PackagesImport {

    public static void main(String[] args) {

        System.out.println("PI = " + PI);

        System.out.println(
            "Square root = " + sqrt(25)
        );
    }
}
```

Output:

```
PI = 3.141592653589793
Square root = 5.0
```

Without static import:

```
System.out.println(Math.PI);
System.out.println(Math.sqrt(25));
```

With static import:

```
System.out.println(PI);
System.out.println(sqrt(25));
```

14. Package Declaration vs Import

These are different.

package	import
Defines the package of the current source file	Makes types from another package conveniently accessible
Usually appears first	Appears after package declaration
Establishes namespace	Does not establish namespace
Example: package com.example.app;	Example: import java.util.List;

Typical source-file structure:

```
package com.example.app;

import java.util.List;
import java.util.ArrayList;

public class PackagesImport {

    // class body
}
```

The order is:

```
package declaration
    ↓
import declarations
    ↓
class/interface/enum declaration
```

15. Java's java.lang Package

The java.lang package is automatically available to every Java source file.

Therefore, you can directly write:

```
String name = "Anurag";
System.out.println(name);
```

without:

```
import java.lang.String;
import java.lang.System;
```

Common java.lang types include:

String
System
Object
Math
Integer
Double
Boolean
Thread
Exception

Conceptually:

```
java.lang
├── String
├── System
├── Object
├── Math
└── Integer
    ↓
Automatically available
```

16. Package and Access Modifiers

Packages are closely related to access control.

For example:

```
class Employee {
    int salary = 50000;
}
```

Because salary has no access modifier, it has **package-private/default access**.

Therefore:

Same package
↓
Can access

Different package

↓

Cannot directly access

This is one reason packages are important in Java's access-control system.

17. Name Conflict Example

Suppose two packages contain classes with the same name:

```
com.company.hr.Employee  
com.company.sales.Employee
```

You cannot simply import both and expect Java to know which Employee you mean:

```
import com.company.hr.Employee;  
import com.company.sales.Employee;
```

Using:

```
Employee employee;
```

would be ambiguous.

Instead, you can use fully qualified names:

```
com.company.hr.Employee hrEmployee =  
    new com.company.hr.Employee();
```

```
com.company.sales.Employee salesEmployee =  
    new com.company.sales.Employee();
```

This demonstrates one of the major purposes of packages:

Packages provide namespaces that help prevent naming conflicts.

18. Complete PackagesImport Example

```
package com.example.app;
```

```
import java.util.ArrayList;  
import java.util.List;
```

```
class PackagesImport {
```

```

public static void main(String[] args) {

    List<String> names =
        new ArrayList<>();

    names.add("Anurag");
    names.add("Rahul");
    names.add("Amit");

    System.out.println("Names:");

    for (String name : names) {

        System.out.println(name);
    }
}

```

Output

Names:
Anurag
Rahul
Amit

Here:

```
package com.example.app;
```

defines the current package.

And:

```
import java.util.ArrayList;
import java.util.List;
```

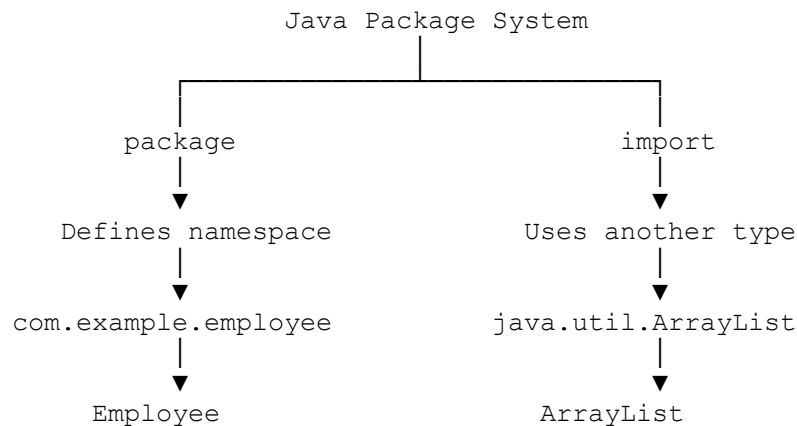
makes the ArrayList and List types available by simple name.

19. Important Rules

Rule	Explanation
package	Declares the package of the source file

import	Makes another package's accessible types convenient to reference
Package declaration	Normally appears before imports
Imports	Appear before top-level type declarations
java.lang	Automatically available
java.util	Not automatically imported
import java.util.*	Imports types directly in java.util, not subpackages
Static import	Imports static members
FQCN	Package + class name
Packages	Provide namespaces and organization
Default access	Limited to the same package

20. Easy Way to Remember



One-line definitions

Package: A namespace mechanism that groups related Java types and helps organize code and control package-level access.

import: A declaration that allows a source file to refer to accessible types or static members by shorter names instead of repeatedly using their fully qualified names.

Java Classes and Objects

Java Version: 1.0

Year: 1996

Feature: Classes and Objects

UniqueClassName: ClassesObjects

1. Definition

Class

A **class** in Java is a **blueprint or template** used to define the data and behavior that objects of that class can have.

A class can contain:

- Fields / instance variables
- Methods
- Constructors
- Static members
- Nested types
- Initializer blocks

Example:

```
class ClassesObjects {  
  
    int id;  
    String name;  
  
    void display() {  
        System.out.println(id + " - " + name);  
    }  
}
```

Here, ClassesObjects is a **class**.

Object

An **object** is a runtime instance of a class.

An object has:

- **State** → represented by fields
- **Behavior** → represented by methods
- **Identity** → distinguishes one object from another

For example:

```
ClassesObjects obj = new ClassesObjects();
```

Here:

ClassesObjects → Class

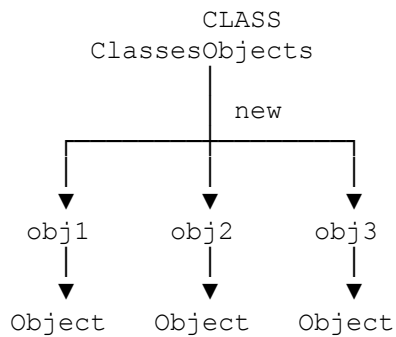
obj → Reference variable

new ClassesObjects() → Object creation

Simple definition

Class = blueprint; Object = runtime instance created from that blueprint.

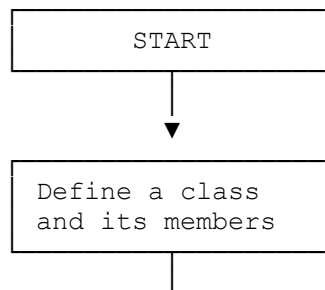
2. Class and Object Relationship

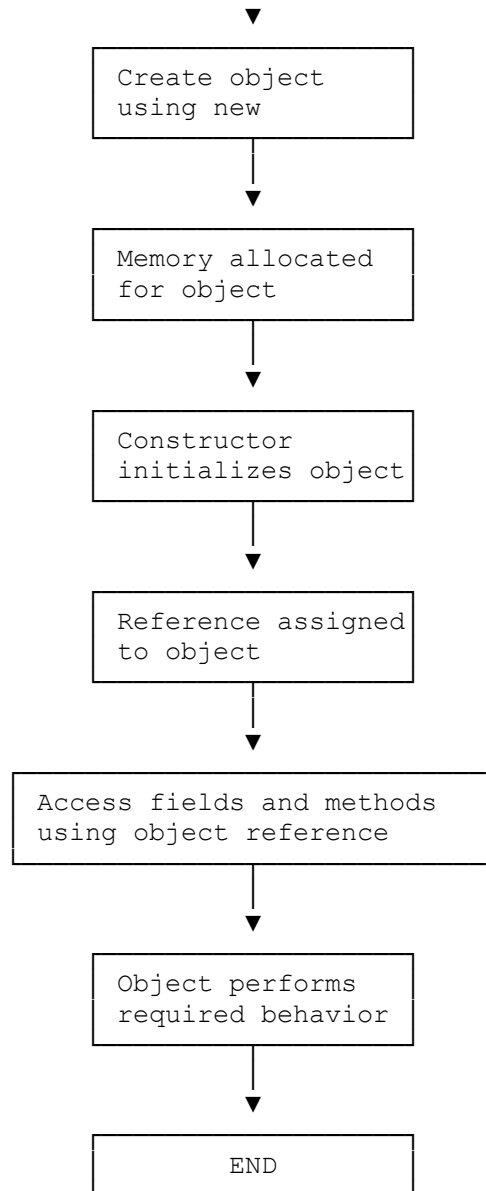


One class can be used to create **many objects**.

Each object generally has its own instance state.

3. Flowchart





4. Basic Java Code Example

```
class ClassesObjects {  
  
    // Instance variables  
    int id;  
    String name;  
  
    // Method  
    void display() {
```

```

        System.out.println("ID    : " + id);
        System.out.println("Name : " + name);
    }

    public static void main(String[] args) {

        // Create object
        ClassesObjects obj =
            new ClassesObjects();

        // Assign object state
        obj.id = 101;
        obj.name = "Anurag";

        // Call method
        obj.display();
    }
}

```

Output

```

ID      : 101
Name    : Anurag

```

5. Understanding the Object-Creation Statement

Consider:

```
ClassesObjects obj = new ClassesObjects();
```

There are three important parts:

```
ClassesObjects obj = new ClassesObjects();
      |               |
      |               |_____ Object creation
      |_____ Reference variable

```

More precisely:

```

ClassesObjects
    |
    ▼
Reference type

obj
|

```

▼
Reference variable

new ClassesObjects()
|
▼
Creates a new object

The reference obj can then be used to access the object's accessible instance members:

obj.id
obj.name
obj.display()

6. Class Contains State and Behavior

A class commonly combines **state** and **behavior**.

```
class ClassesObjects {  
  
    // State  
    int id;  
    String name;  
  
    // Behavior  
    void display() {  
  
        System.out.println(  
            id + " - " + name  
        );  
    }  
}
```

Here:

Component	Example	Represents
Field	id	State
Field	name	State
Method	display()	Behavior
Class	ClassesObjects	Blueprint

7. Creating Multiple Objects

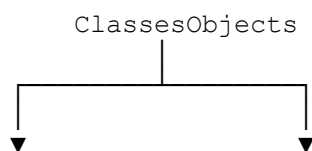
One class can create multiple objects.

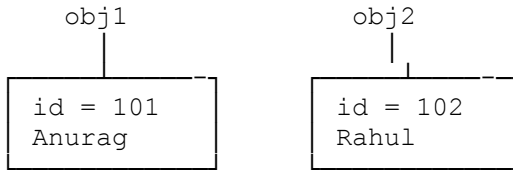
```
class ClassesObjects {  
  
    int id;  
    String name;  
  
    void display() {  
  
        System.out.println(  
            id + " - " + name  
        );  
    }  
  
    public static void main(String[] args) {  
  
        ClassesObjects obj1 =  
            new ClassesObjects();  
  
        ClassesObjects obj2 =  
            new ClassesObjects();  
  
        obj1.id = 101;  
        obj1.name = "Anurag";  
  
        obj2.id = 102;  
        obj2.name = "Rahul";  
  
        obj1.display();  
        obj2.display();  
    }  
}
```

Output

101 - Anurag
102 - Rahul

Conceptually:





Each object has its own instance state.

8. Class with Constructor

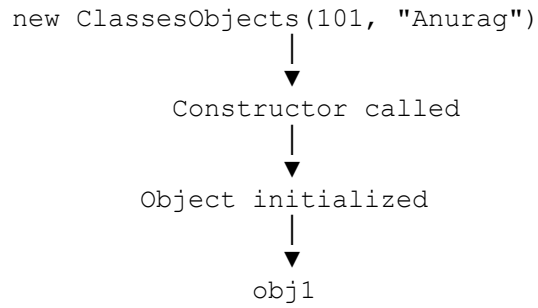
A constructor can initialize an object's state when the object is created.

```
class ClassesObjects {  
  
    int id;  
    String name;  
  
    // Constructor  
    ClassesObjects(int id, String name) {  
  
        this.id = id;  
        this.name = name;  
    }  
  
    void display() {  
  
        System.out.println(  
            id + " - " + name  
        );  
    }  
  
    public static void main(String[] args) {  
  
        ClassesObjects obj1 =  
            new ClassesObjects(101, "Anurag");  
  
        ClassesObjects obj2 =  
            new ClassesObjects(102, "Rahul");  
  
        obj1.display();  
        obj2.display();  
    }  
}
```

Output:

```
101 - Anurag  
102 - Rahul
```

Flow:



9. Object State and Behavior

Consider:

```
ClassesObjects obj =
    new ClassesObjects(101, "Anurag");
```

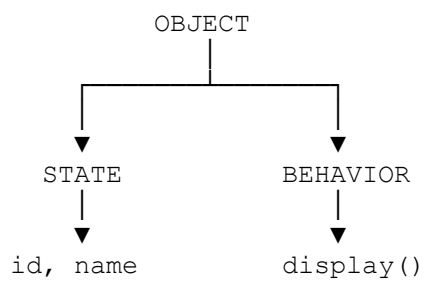
The object's state is:

```
id = 101
name = "Anurag"
```

Its behavior is provided by:

```
obj.display();
```

So:



10. Object Identity

Two objects created from the same class are separate objects.

```
ClassesObjects obj1 =  
    new ClassesObjects(101, "Anurag");
```

```
ClassesObjects obj2 =  
    new ClassesObjects(101, "Anurag");
```

Although their data is initially the same:

```
obj1 → id=101, name=Anurag  
obj2 → id=101, name=Anurag
```

they are still two different object instances.

```
System.out.println(obj1 == obj2);
```

Output:

false

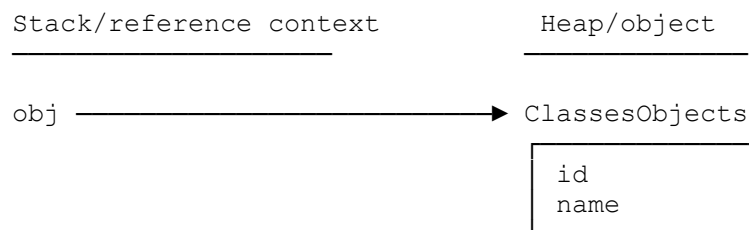
The == comparison between object references checks whether both references refer to the **same object**.

11. Reference Variable vs Object

This distinction is important.

```
ClassesObjects obj =  
    new ClassesObjects();
```

Conceptually:



obj is a **reference variable**.

The new ClassesObjects() expression creates an **object**.

The exact JVM memory implementation is more nuanced than this simplified diagram, but this model is useful for learning the distinction.

12. Null Reference

A reference variable can contain null.

```
class ClassesObjects {  
    public static void main(String[] args) {  
        ClassesObjects obj = null;  
        System.out.println(obj);  
    }  
}
```

Output:

null

But this would cause a NullPointerException:

```
obj.display();
```

because obj does not currently refer to an object.

13. Complete Java Example

```
class ClassesObjects {  
    // Instance fields  
    private int id;  
    private String name;  
  
    // Constructor  
    ClassesObjects(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
  
    // Instance method  
    void display() {
```

```

        System.out.println("ID    : " + id);
        System.out.println("Name : " + name);
    }

    // Instance method
    void greet() {

        System.out.println(
            "Hello, " + name + "!"
        );
    }

    public static void main(String[] args) {

        // Object 1
        ClassesObjects obj1 =
            new ClassesObjects(101, "Anurag");

        // Object 2
        ClassesObjects obj2 =
            new ClassesObjects(102, "Rahul");

        // Object 1 behavior
        obj1.display();
        obj1.greet();

        System.out.println();

        // Object 2 behavior
        obj2.display();
        obj2.greet();
    }
}

```

Output

```

ID      : 101
Name    : Anurag
Hello, Anurag!

```

```

ID      : 102
Name    : Rahul
Hello, Rahul!

```

14. Class vs Object

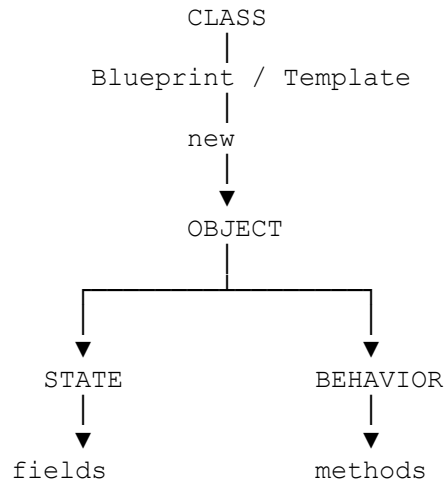
Feature	Class	Object
---------	-------	--------

Meaning	Blueprint/template	Runtime instance
Exists as a type	Yes	No, it is an instance
Defines fields	Yes	Contains field values
Defines methods	Yes	Uses those methods
Memory for instance fields	Allocated per object	Has its own instance state
Number	One class can create many objects	Each object is a separate instance
Creation	Declared in source code	Usually created with new
Example	class Car {}	new Car()

15. Class, Object, Reference — Difference

Term	Meaning	Example
Class	Defines structure and behavior	ClassesObjects
Object	Runtime instance	new ClassesObjects()
Reference	Variable that refers to an object	Obj
Constructor	Initializes object during creation	ClassesObjects(...)
Field	Stores object state	id, name
Method	Defines object behavior	display()

16. Easy Way to Remember



One-line definition

A class is a blueprint that defines the state and behavior of objects, while an object is a runtime instance of that class with its own identity and instance state.

Java Encapsulation

Java Version: 1.0

Year: 1996

Feature: Encapsulation

UniqueClassName: Encapsulation

1. Definition

Encapsulation is an object-oriented programming principle in which **data (fields) and the methods that operate on that data are bundled together inside a class**, while direct access to the internal state can be restricted using access modifiers.

In Java, encapsulation is commonly implemented by:

1. Declaring fields as private.
2. Providing controlled public methods to read or modify those fields.
3. Validating or controlling changes inside those methods.

Simple definition

Encapsulation means bundling data and behavior together and controlling direct access to the object's internal state.

For example:

```
class Encapsulation {  
    private int age;  
  
    public int getAge() {  
        return age;  
    }  
  
    public void setAge(int age) {  
        if (age >= 0) {  
            this.age = age;  
        }  
    }  
}
```

Here:

```
private age  
↓  
Direct external access restricted  
↓  
getAge() / setAge()  
↓  
Controlled access
```

2. Why Encapsulation Is Needed

Without encapsulation:

```
class Student {  
    public int age;  
}
```

External code can directly assign invalid values:

```
Student student = new Student();
```

```
student.age = -100;
```

There is no validation.

With encapsulation:

```
class Student {  
    private int age;  
  
    public void setAge(int age) {  
        if (age >= 0) {  
            this.age = age;  
        }  
    }  
}
```

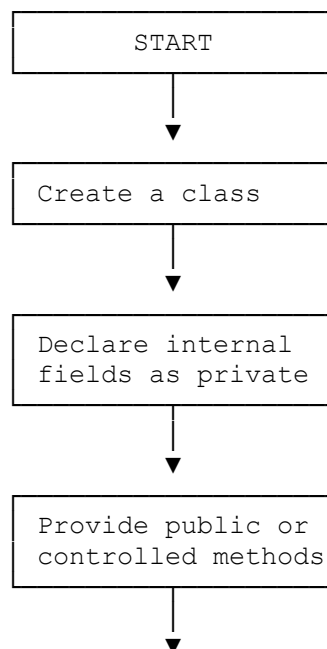
Now:

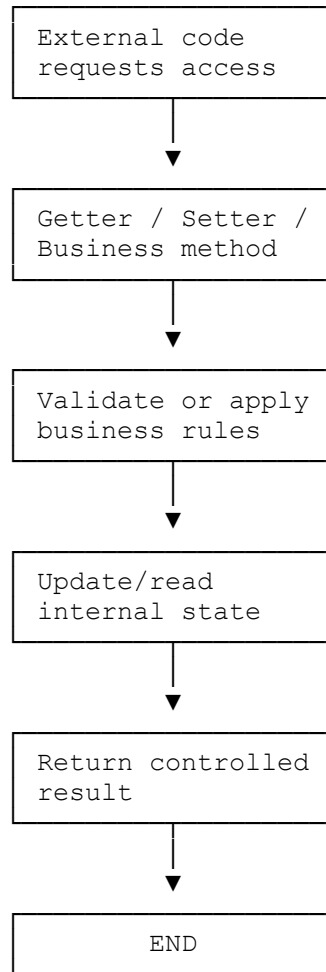
```
student.setAge(-100);
```

can be rejected by the class.

Thus, the class controls how its internal state changes.

3. Encapsulation Flowchart





4. Basic Java Code Example

```
class Encapsulation {  
  
    // Private fields  
    private int id;  
    private String name;  
  
    // Setter for id  
    public void setId(int id) {  
  
        if (id > 0) {  
            this.id = id;  
        }  
    }  
  
    // Getter for id
```

```

    public int getId() {

        return id;
    }

    // Setter for name
    public void setName(String name) {

        if (name != null && !name.isBlank()) {
            this.name = name;
        }
    }

    // Getter for name
    public String getName() {

        return name;
    }

    public static void main(String[] args) {

        Encapsulation obj =
            new Encapsulation();

        obj.setId(101);
        obj.setName("Anurag");

        System.out.println("ID    : " + obj.getId());
        System.out.println("Name  : " + obj.getName());
    }
}

```

Output

```

ID      : 101
Name    : Anurag

```

5. How the Example Works

The field:

```
private int id;
```

cannot be directly accessed from outside the class.

This is invalid:

```
obj.id = 101;
```

Instead, external code calls:

```
obj.setId(101);
```

The setter controls the assignment:

```
public void setId(int id) {  
    if (id > 0) {  
        this.id = id;  
    }  
}
```

Similarly, reading the value is controlled through:

```
obj.getId();
```

6. Encapsulation with Validation

One of the major benefits of encapsulation is that the class can enforce rules.

```
class Encapsulation {  
    private double balance;  
  
    public void deposit(double amount) {  
        if (amount > 0) {  
            balance += amount;  
  
            System.out.println(  
                "Deposit successful"  
            );  
        } else {  
            System.out.println(  
                "Invalid deposit amount"  
            );  
        }  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
  
    public static void main(String[] args) {
```

```

        Encapsulation account =
            new Encapsulation();

        account.deposit(5000);
        account.deposit(-100);

        System.out.println(
            "Balance: " + account.getBalance()
        );
    }
}

```

Output

Deposit successful
 Invalid deposit amount
 Balance: 5000.0

The caller cannot directly do:

```
account.balance = -5000;
```

because balance is private.

7. Encapsulation Without Traditional Getters/Setters

Encapsulation does **not** require that every private field have a getter and setter.

A better design sometimes exposes **business operations** instead.

For example:

```

class BankAccount {
    private double balance;

    public void deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException(
                "Amount must be positive"
            );
        }

        balance += amount;
    }
}

```

```

public void withdraw(double amount) {

    if (amount <= 0 || amount > balance) {
        throw new IllegalArgumentException(
            "Invalid withdrawal"
        );
    }

    balance -= amount;
}

public double getBalance() {

    return balance;
}
}

```

This is stronger encapsulation because external code cannot arbitrarily set:

```
balance = 1000000;
```

Instead, it must use valid operations:

```
account.deposit(5000);
account.withdraw(1000);
```

8. Complete Real-World Example

```

class Encapsulation {

    private long accountNumber;
    private String accountHolder;
    private double balance;

    public Encapsulation(
        long accountNumber,
        String accountHolder,
        double initialBalance) {

        if (accountNumber <= 0) {
            throw new IllegalArgumentException(
                "Invalid account number"
            );
        }

        if (accountHolder == null ||
            accountHolder.isBlank()) {

```

```

        throw new IllegalArgumentException(
            "Invalid account holder"
        );
    }

    if (initialBalance < 0) {
        throw new IllegalArgumentException(
            "Initial balance cannot be negative"
        );
    }

    this.accountNumber = accountNumber;
    this.accountHolder = accountHolder;
    this.balance = initialBalance;
}

public void deposit(double amount) {

    if (amount <= 0) {
        throw new IllegalArgumentException(
            "Deposit must be positive"
        );
    }

    balance += amount;
}

public void withdraw(double amount) {

    if (amount <= 0) {
        throw new IllegalArgumentException(
            "Withdrawal must be positive"
        );
    }

    if (amount > balance) {
        throw new IllegalArgumentException(
            "Insufficient balance"
        );
    }

    balance -= amount;
}

public long getAccountNumber() {

    return accountNumber;
}

public String getAccountHolder() {

    return accountHolder;
}

public double getBalance() {

    return balance;
}

```

```

    }

    public static void main(String[] args) {

        Encapsulation account =
            new Encapsulation(
                123456789L,
                "Anurag",
                10000
            );

        account.deposit(5000);
        account.withdraw(2000);

        System.out.println(
            "Account Number: " +
            account.getAccountNumber()
        );

        System.out.println(
            "Account Holder: " +
            account.getAccountHolder()
        );

        System.out.println(
            "Balance: " +
            account.getBalance()
        );
    }
}

```

Output

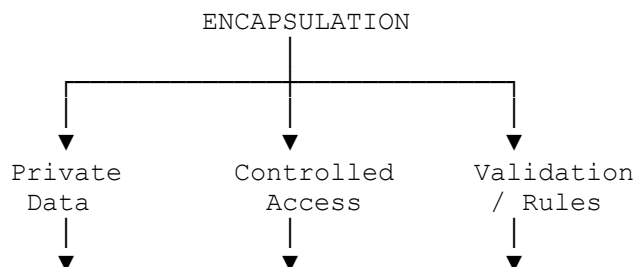
```

Account Number : 123456789
Account Holder  : Anurag
Balance         : 13000.0

```

9. Four Important Components

Encapsulation can be understood through four concepts:



fields methods/API business logic

For example:

```
private double balance;
```

is the **hidden state**.

```
public void deposit(double amount)
```

is the **controlled operation**.

```
if (amount > 0)
```

is the **validation/business rule**.

10. Encapsulation and Access Modifiers

Access modifiers provide the mechanisms needed to control visibility.

Modifier	Role in Encapsulation
private	Strongly restricts direct member access
public	Exposes controlled operations/API
protected	Allows package/subclass access
default	Restricts access to the package

A common pattern is:

```
class User {  
  
    private String username;  
    private String password;  
  
    public String getUsername() {  
        return username;  
    }  
  
    public void setUsername(String username) {  
        this.username = username;  
    }  
}
```

```
}  
}
```

The fields are hidden while selected operations are exposed.

11. Encapsulation vs Data Hiding

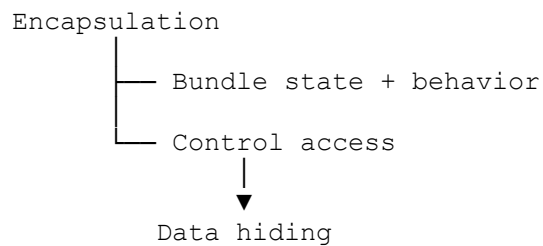
These terms are related but not exactly identical.

Encapsulation

Focuses on **bundling data and behavior together** and controlling how the object's state is accessed or modified.

Data hiding

Focuses specifically on **restricting direct access to internal implementation details**.



In Java, private fields are a common way to implement data hiding within an encapsulated class.

12. Benefits of Encapsulation

Benefit	Explanation
Data protection	Prevents uncontrolled direct modification
Validation	Allows rules to be enforced
Maintainability	Internal implementation can change without changing the public API
Loose coupling	Clients depend on exposed operations rather

	than internal representation
Security	Sensitive state can be restricted
Flexibility	Implementation can change behind the same interface
Consistency	Object state can be kept valid
Abstraction support	Hides unnecessary implementation details

13. Without Encapsulation vs With Encapsulation

Without encapsulation

```
class Employee {
    public double salary;
}
```

External code:

```
employee.salary = -50000;
```

There is no protection.

With encapsulation

```
class Employee {
    private double salary;

    public void setSalary(double salary) {
        if (salary >= 0) {
            this.salary = salary;
        }
    }

    public double getSalary() {
        return salary;
    }
}
```

Now:

```
employee.setSalary(-50000);
```

can be rejected.

14. Important Point: Encapsulation Is More Than Getters and Setters

This is a common misconception.

Simply writing:

```
private int value;
```

```
public int getValue() {  
    return value;  
}
```

```
public void setValue(int value) {  
    this.value = value;  
}
```

provides controlled access, but good encapsulation is about **protecting invariants and exposing an appropriate API**, not mechanically generating getters and setters for every field.

For example, instead of:

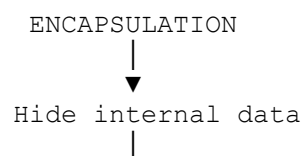
```
account.setBalance(100000);
```

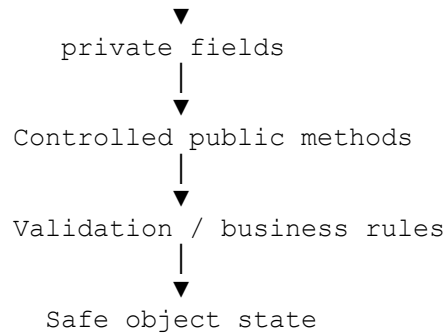
a better domain API may be:

```
account.deposit(100000);
```

because the class controls how the state changes.

15. Easy Way to Remember





One-line definition

Encapsulation is the OOP principle of bundling an object's state and behavior together while restricting and controlling direct access to its internal state.

Java Inheritance

Java Version: 1.0

Year: 1996

Feature: Inheritance

UniqueClassName: Inheritance

1. Definition

Inheritance is an object-oriented programming mechanism in Java through which one class (**subclass/child class**) acquires accessible fields and methods from another class (**superclass/parent class**).

Inheritance is primarily implemented using the `extends` keyword for classes.

Simple definition

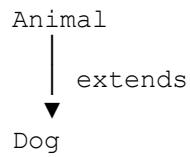
Inheritance allows a child class to reuse and extend the accessible state and behavior of a parent class.

Example:

```
class Animal {  
  
    void eat() {  
        System.out.println("Animal eats");  
    }  
}
```

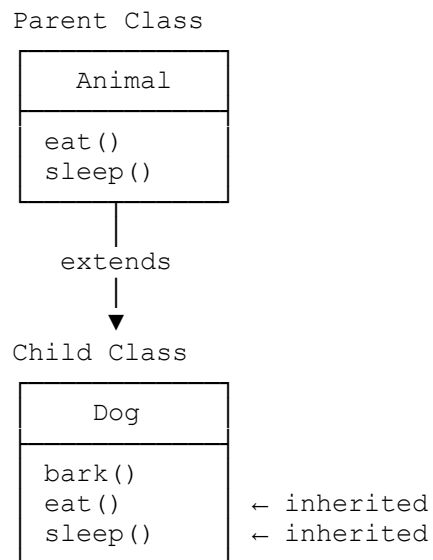
```
}  
  
class Dog extends Animal {  
  
    void bark() {  
        System.out.println("Dog barks");  
    }  
}
```

Here:

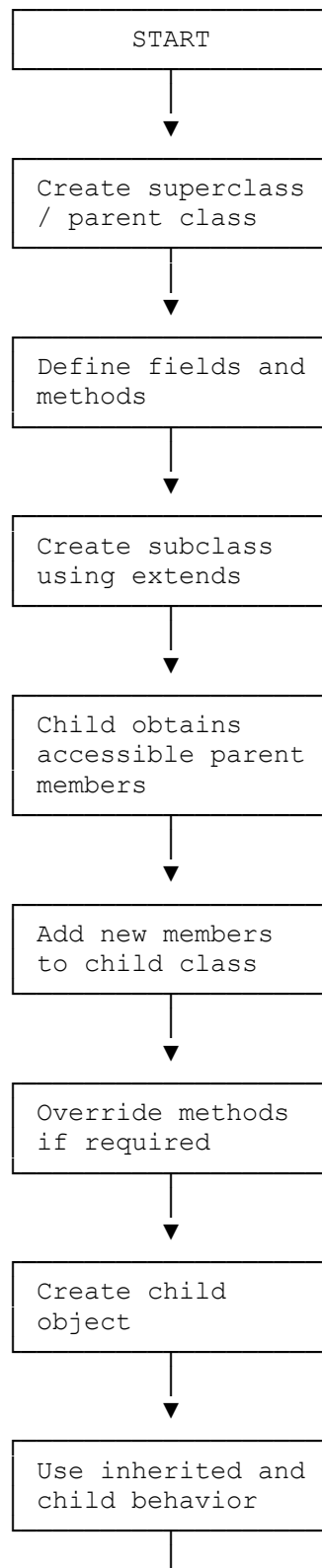


Dog inherits the accessible eat() method from Animal and adds its own bark() method.

2. Basic Inheritance Structure



3. Inheritance Flowchart



▼

END

4. Basic Java Code Example

```
class Animal {  
    void eat() {  
        System.out.println("Animal eats");  
    }  
    void sleep() {  
        System.out.println("Animal sleeps");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog barks");  
    }  
  
    public static void main(String[] args) {  
        Dog dog = new Dog();  
  
        // Inherited methods  
        dog.eat();  
        dog.sleep();  
  
        // Dog's own method  
        dog.bark();  
    }  
}
```

Output

Animal eats
Animal sleeps
Dog barks

5. Understanding extends

The keyword:

extends

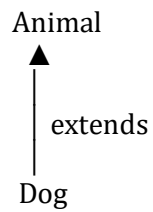
establishes a class inheritance relationship.

```
class Dog extends Animal {  
}
```

means:

Dog IS-A Animal

Conceptually:



Therefore, a Dog object can use the accessible inherited behavior of Animal.

6. Parent and Child Class

Java terminology:

Term	Meaning
Superclass	Parent/base class
Subclass	Child/derived class
Parent class	Class being inherited from
Child class	Class that inherits
extends	Keyword used for class inheritance

Example:

```
class Vehicle {  
}
```

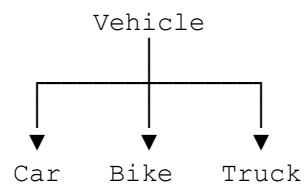
is the superclass.

```
class Car extends Vehicle {  
}
```

is the subclass.

7. Real-World Example

Consider:



A vehicle may have common behavior:

```
class Vehicle {  
  
    void start() {  
  
        System.out.println(  
            "Vehicle starts"  
        );  
    }  
  
    void stop() {  
  
        System.out.println(  
            "Vehicle stops"  
        );  
    }  
}
```

A car can inherit it:

```
class Car extends Vehicle {
```

```

void drive() {

    System.out.println(
        "Car drives"
    );
}
}

```

Complete example:

```

class Inheritance {

    static class Vehicle {

        void start() {
            System.out.println(
                "Vehicle starts"
            );
        }

        void stop() {
            System.out.println(
                "Vehicle stops"
            );
        }
    }

    static class Car extends Vehicle {

        void drive() {
            System.out.println(
                "Car drives"
            );
        }
    }

    public static void main(String[] args) {

        Car car = new Car();

        car.start();
        car.drive();
        car.stop();
    }
}

```

Output

```

Vehicle starts
Car drives
Vehicle stops

```

8. Inheritance and Code Reuse

Without inheritance:

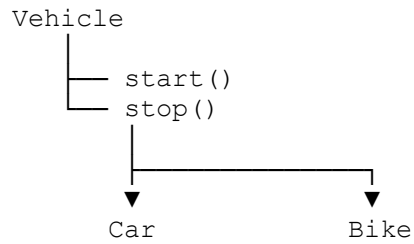
```
class Car {  
  
    void start() {  
        System.out.println("Vehicle starts");  
    }  
  
    void stop() {  
        System.out.println("Vehicle stops");  
    }  
}  
  
class Bike {  
  
    void start() {  
        System.out.println("Vehicle starts");  
    }  
  
    void stop() {  
        System.out.println("Vehicle stops");  
    }  
}
```

The common behavior is duplicated.

With inheritance:

```
class Vehicle {  
  
    void start() {  
        System.out.println("Vehicle starts");  
    }  
  
    void stop() {  
        System.out.println("Vehicle stops");  
    }  
}  
  
class Car extends Vehicle {  
}  
  
class Bike extends Vehicle {  
}
```

Now:



Both classes can reuse the common behavior.

9. Inheritance with Fields

Inheritance applies to accessible inherited members, not just methods.

```
class Person {

    String name;
    int age;

    void displayPerson() {

        System.out.println(
            "Name: " + name
        );

        System.out.println(
            "Age: " + age
        );
    }
}

class Student extends Person {

    int rollNumber;

    void displayStudent() {

        System.out.println(
            "Roll Number: " + rollNumber
        );
    }
}
```

Usage:

```
class Inheritance {  
  
    public static void main(String[] args) {  
  
        Student student = new Student();  
  
        student.name = "Anurag";  
        student.age = 25;  
        student.rollNumber = 101;  
  
        student.displayPerson();  
        student.displayStudent();  
    }  
}
```

Output:

```
Name           : Anurag  
Age            : 25  
Roll Number    : 101
```

10. Inheritance and Method Overriding

A child class can provide its own implementation of an inherited instance method. This is called **method overriding** when the Java overriding rules are satisfied.

```
class Animal {  
  
    void sound() {  
  
        System.out.println(  
            "Animal makes a sound"  
        );  
    }  
}
```

```
class Dog extends Animal {  
  
    @Override  
    void sound() {  
  
        System.out.println(  
            "Dog barks"  
        )  
    }  
}
```

```
    );  
  }  
}
```

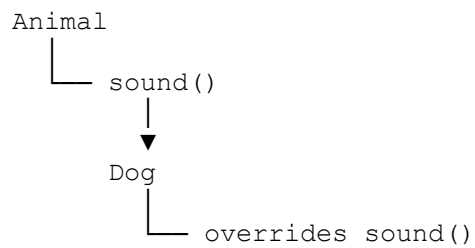
Now:

```
class Inheritance {  
    public static void main(String[] args) {  
        Dog dog = new Dog();  
        dog.sound();  
    }  
}
```

Output:

Dog barks

Conceptually:



11. super and Inheritance

The `super` keyword is used to refer to the immediate superclass in contexts where it is applicable.

For example:

```
class Animal {  
    void sound() {  
        System.out.println(  
            "Animal sound"  
        );  
    }  
}
```

```
class Dog extends Animal {  
  
    @Override  
    void sound() {  
  
        super.sound();  
  
        System.out.println(  
            "Dog barks"  
        );  
    }  
}
```

Output:

Animal sound
Dog barks

Here:

```
super.sound();
```

calls the superclass implementation.

12. Constructor and Inheritance

Constructors are **not inherited**.

However, when a subclass object is created, superclass initialization occurs as part of object construction.

Example:

```
class Parent {  
  
    Parent() {  
  
        System.out.println(  
            "Parent constructor"  
        );  
    }  
}  
  
class Child extends Parent {  
  
    Child() {
```

```

        System.out.println(
            "Child constructor"
        );
    }
}

class Inheritance {

    public static void main(String[] args) {

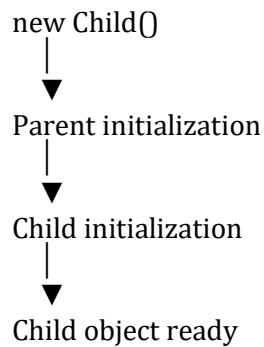
        Child obj = new Child();
    }
}

```

Output:

Parent constructor
Child constructor

Conceptually:



If the superclass constructor invocation is not explicitly written, Java inserts an implicit `super()` when applicable.

13. Types of Class Inheritance in Java

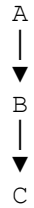
Java supports these class-inheritance structures:

Single inheritance



```
class B extends A {  
}
```

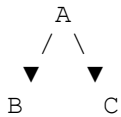
Multilevel inheritance



```
class B extends A {  
}
```

```
class C extends B {  
}
```

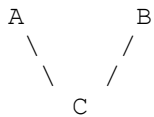
Hierarchical inheritance



```
class B extends A {  
}
```

```
class C extends A {  
}
```

Java **does not support multiple inheritance of classes:**



You cannot write:

```
class C extends A, B {  
}
```

This is illegal Java syntax.

However, Java supports multiple inheritance of **type through interfaces**:

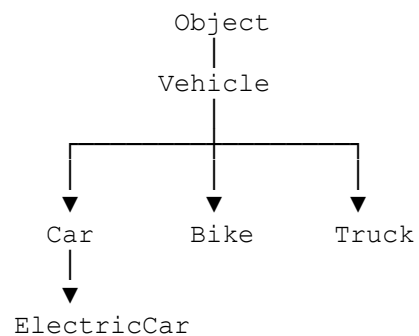
```
interface A {  
}
```

```
interface B {  
}
```

```
class C implements A, B {  
}
```

14. Inheritance Hierarchy

A larger hierarchy can look like:



For example:

```
class Vehicle {  
}
```

```
class Car extends Vehicle {  
}
```

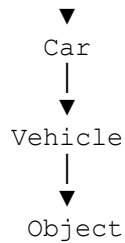
```
class Bike extends Vehicle {  
}
```

```
class Truck extends Vehicle {  
}
```

```
class ElectricCar extends Car {  
}
```

This creates:

```
ElectricCar  
|
```



Every Java class ultimately has Object as an ancestor, directly or indirectly, except that Object itself has no superclass.

15. Access Modifiers and Inheritance

Not every member of a superclass is directly accessible in a subclass.

Superclass Member	Directly Accessible in Subclass?
public	✓
protected	✓, subject to access rules
default/package-private	✓ only when package access permits
private	✗ directly

For example:

```

class Parent {
    public int a = 10;
    protected int b = 20;
    int c = 30;
    private int d = 40;
}

class Child extends Parent {
    void display() {
        System.out.println(a);    // ✓
        System.out.println(b);    // ✓
        System.out.println(c);    // depends on package
        // System.out.println(d); // ✗
    }
}
  
```

The private member still belongs to the superclass's object state, but the subclass cannot directly access it by its name.

16. IS-A Relationship

Inheritance represents an **IS-A relationship**.

Examples:

```
Dog IS-A Animal
Car IS-A Vehicle
Student IS-A Person
Manager IS-A Employee
```

Therefore:

```
class Dog extends Animal {
}
```

means:

Dog IS-A Animal

This is different from a **HAS-A relationship**, which is generally modeled using composition.

Example:

```
class Car {
    Engine engine;
}
```

This means:

Car HAS-A Engine

17. Inheritance vs Composition

Inheritance	Composition
-------------	-------------

Represents IS-A	Represents HAS-A
Uses extends	Uses object references/fields
Creates class hierarchy	Combines objects
Promotes subtype polymorphism	Promotes flexible object composition
Example: Dog extends Animal	Example: Car has an Engine

18. Complete Example with Inheritance + Overriding

```

class Inheritance {
    static class Animal {
        protected String name;

        Animal(String name) {
            this.name = name;
        }

        void sound() {
            System.out.println(
                name + " makes a sound"
            );
        }

        void eat() {
            System.out.println(
                name + " eats"
            );
        }
    }

    static class Dog extends Animal {
        Dog(String name) {
            super(name);
        }

        @Override
        void sound() {

```

```

        System.out.println(
            name + " barks"
        );
    }

    void guard() {

        System.out.println(
            name + " guards the house"
        );
    }
}

public static void main(String[] args) {

    Dog dog =
        new Dog("Bruno");

    dog.eat();        // inherited
    dog.sound();      // overridden
    dog.guard();      // child-specific
}
}

```

Output

Bruno eats
 Bruno barks
 Bruno guards the house

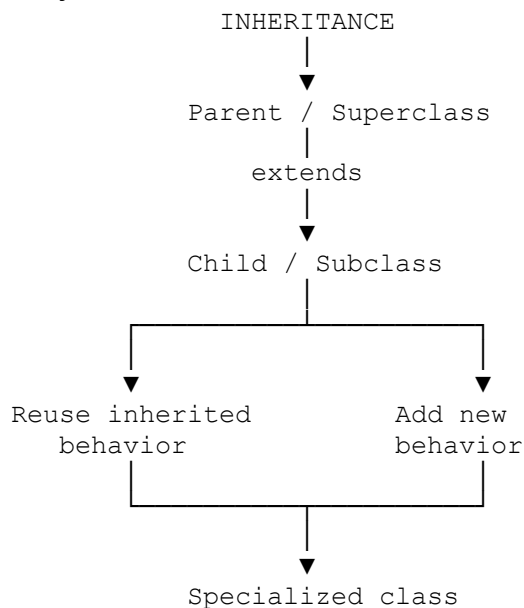
19. Major Benefits of Inheritance

Benefit	Explanation
Code reuse	Common functionality can be defined in a superclass
Extensibility	Subclasses can add specialized behavior
Method overriding	Subclasses can provide specialized implementations
Polymorphism	Parent references can represent compatible subclass objects
Maintainability	Common functionality can be centralized
Hierarchical modeling	Real-world IS-A relationships can be represented

20. Important Rules

1. Use extends for class inheritance.
 2. A Java class can directly extend **only one class**.
 3. Constructors are **not inherited**.
 4. Private superclass members are not directly accessible in a subclass.
 5. protected provides special package/subclass access.
 6. A subclass can add its own fields and methods.
 7. A subclass can override eligible inherited instance methods.
 8. super can access the immediate superclass's members and constructors where permitted.
 9. Every ordinary Java class ultimately derives from Object.
 10. Multiple inheritance of classes is not supported.
 11. Multiple interfaces can be implemented by a class.
 12. Inheritance generally represents an **IS-A relationship**.
-

21. Easy Way to Remember



One-line definition

Inheritance is the Java mechanism by which a subclass derives from a superclass, gaining access to inherited members and allowing the subclass to reuse, specialize, or extend the superclass's behavior.